

SOLIDWORKS API (.NET VBA C#) — Examples (24/50):

Get_Area_Hatch_Data_Example_CSharp

Get_Centerline_Annotation_Information_Ex

SOLIDWORKS API Documentation (converted from CHM)

Get Area Hatch Data Example (C#)

This example shows how to get the data for an area hatch in a drawing.

```
//-----
// Preconditions:
// 1. Verify that the specified drawing exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the drawing.
// 2. Hatches a face in the drawing.
// 3. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDraw;
        SelectionMgr swSelMgr;
        View swView;
        Sketch swSketch;
        object[] vSketchHatch;
        SketchHatch swSketchHatch;
        Face2 swFace;
        int[] vID;
        int i;
        bool bRet;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\box.slddrw", (int)swDocumentTypes_e.swDocDRAWING, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("box - Sheet1", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;

            bRet = swDraw.ActivateView("Drawing View1");

            bRet = swModel.Extension.SelectByID("", "FACE", 0.246685509728212, 0.236217308689246, 0.0149999999999864, true, 0, null);
            swModel.InsertHatchedFace();
            swModel.ClearSelection2(true);

            bRet = swModel.Extension.SelectByID("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            swView.ScaleHatchPattern = true;
            swSketch = (Sketch)swView.GetSketch();
            swModel.EditSketch();

            swModel.ClearSelection2(true);

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swView.Name);

            vSketchHatch = (object[])swSketch.GetSketchHatches();

            if ((vSketchHatch != null))
            {
                for (i = 0; i <= vSketchHatch.GetUpperBound(0); i++)
                {
                    swSketchHatch = (SketchHatch)vSketchHatch[i];
                    swFace = (Face2)swSketchHatch.GetFace();

                    bRet = swSketchHatch.Select4(true, null);
                    vID = (int[])swSketchHatch.GetID();

                    Debug.Print("    HatchID(" + i + "): [" + vID[0] + ", " + vID[1] + "]");
                    Debug.Print("    Angle: " + swSketchHatch.Angle);
                    Debug.Print("    Color: " + swSketchHatch.Color);
                    Debug.Print("    Layer: " + swSketchHatch.Layer);
                }
            }
        }
    }
}
```

```

        Debug.Print("      Layer override? " + swSketchHatch.LayerOverride);
        Debug.Print("      Pattern: " + swSketchHatch.Pattern);
        Debug.Print("      Scale: " + swSketchHatch.Scale2);
        Debug.Print("      Solid fill? " + swSketchHatch.SolidFill);
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Area Hatch Data Example (VBA)

This example shows how to get the data for an area hatch in a drawing.

```

'-----
' Preconditions:
' 1. Verify that the specified drawing exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Hatches a face in the drawing.
' 3. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim swSketch As SldWorks.Sketch
Dim vSketchHatch As Variant
Dim swSketchHatch As SldWorks.SketchHatch
Dim swFace As SldWorks.Face2
Dim vID As Variant
Dim i As Long
Dim bRet As Boolean
Dim lngstatus As Long, lngwarnings As Long

Option Explicit

Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.slddrw",
swDocumentTypes_e.swDocDRAWING, 0, "", lngstatus, lngwarnings)
    swApp.ActivateDoc2 "box - Sheet1", False, lngstatus
    Set swModel = swApp.ActiveDoc

    bRet = swModel.ActivateView("Drawing View1")

    bRet = swModel.Extension.SelectByID("", "FACE", 0.246685509728212, 0.236217308689246, 1.49999999999864E-02, True, 0, Nothing,
0)
    swModel.InsertHatchedFace
    swModel.ClearSelection2 True

    bRet = swModel.Extension.SelectByID("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    swView.ScaleHatchPattern = True
    Set swSketch = swView.GetSketch
    swModel.EditSketch

    swModel.ClearSelection2 True

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swView.Name

    vSketchHatch = swSketch.GetSketchHatches
    If Not IsEmpty(vSketchHatch) Then
        For i = 0 To UBound(vSketchHatch)
            Set swSketchHatch = vSketchHatch(i)
            Set swFace = swSketchHatch.GetFace

            bRet = swSketchHatch.Select4(True, Nothing)
            vID = swSketchHatch.GetID

            Debug.Print "    HatchID(" & i & "): [" & vID(0) & ", " & vID(1) & "]"
            Debug.Print "    Angle: " & swSketchHatch.Angle
            Debug.Print "    Color: " & swSketchHatch.Color
            Debug.Print "    Layer: " & swSketchHatch.Layer
            Debug.Print "    Layer override? " & swSketchHatch.LayerOverride
            Debug.Print "    Pattern: " & swSketchHatch.Pattern
            Debug.Print "    Scale: " & swSketchHatch.Scale2
            Debug.Print "    Solid fill? " & swSketchHatch.SolidFill

        Next i
    End If
End Sub

```

Get Area Hatch Data Example (VB.NET)

This example shows how to get the data for an area hatch in a drawing.

```

'-----
' Preconditions:
' 1. Verify that the specified drawing exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Hatches a face in the drawing.
' 3. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swView As View
    Dim swSketch As Sketch
    Dim vSketchHatch As Object
    Dim swSketchHatch As SketchHatch
    Dim swFace As Face2
    Dim vID As Object
    Dim i As Integer
    Dim bRet As Boolean
    Dim lngstatus As Integer, lngwarnings As Integer

    Sub main()

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\box.slddrw", swDocumentTypes_e.swDocDRAWING, 0, "", lngstatus, lngwarnings)
        swApp.ActivateDoc2("box - Sheet1", False, lngstatus)
        swModel = swApp.ActiveDoc

        bRet = swModel.Extension.SelectByID2("", "FACE", 0.246685509728212, 0.236217308689246, 0.01499999999999864, True, 0, Nothing,
        swModel.InsertHatchedFace()
        swModel.ClearSelection2(True)

        bRet = swModel.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)

        swSelMgr = swModel.SelectionManager
        swView = swSelMgr.GetSelectedObject6(1, -1)
        swView.ScaleHatchPattern = True
        swSketch = swView.GetSketch
        swModel.EditSketch()

        swModel.ClearSelection2(True)

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swView.Name)

        vSketchHatch = swSketch.GetSketchHatches

        If Not IsNothing(vSketchHatch) Then

            For i = 0 To UBound(vSketchHatch)

                swSketchHatch = vSketchHatch(i)
                swFace = swSketchHatch.GetFace

                bRet = swSketchHatch.Select4(True, Nothing)
                vID = swSketchHatch.GetID

                Debug.Print("    HatchID(" & i & "): [" & vID(0) & ", " & vID(1) & "]")
                Debug.Print("    Angle: " & swSketchHatch.Angle)
                Debug.Print("    Color: " & swSketchHatch.Color)
                Debug.Print("    Layer: " & swSketchHatch.Layer)
                Debug.Print("    Layer override? " & swSketchHatch.LayerOverride)
                Debug.Print("    Pattern: " & swSketchHatch.Pattern)
                Debug.Print("    Scale: " & swSketchHatch.Scale2)
                Debug.Print("    Solid fill? " & swSketchHatch.SolidFill)

            Next i

        End If

    End Sub

''' <summary>

```

```
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Get Areas of MidSurface Faces Example (C#)

This example shows how to get the areas of mid-surface faces.

```
//-----
// Postconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects two faces and inserts a midsurface feature.
// 3. Gets the number of faces in the midsurface feature.
// 4. Gets the areas of the faces in the midsurface feature.
// 5. Examine the Immediate window, FeatureManager design
//    tree, and graphics area.
//
// NOTE: Because the part used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            MidSurface3 swMidSurfaceFeature = default(MidSurface3);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Face2 swFace = default(Face2);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int count = 0;
            object[] faces = null;
            int i = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", -0.0533080255641494, 0.02999999999999727, 0.0131069871973182, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0370905424398416, 0, 0.0289438729892595, true, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeatureManager.InsertMidSurface(null, null, 0.0, false);
            status = swModelDocExt.SelectByID2("Surface-MidSurface1", "REFSURFACE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swMidSurfaceFeature = (MidSurface3)swFeature.GetSpecificFeature2();
            count = swMidSurfaceFeature.GetFaceCount();
            Debug.Print("Number of faces for midsurface feature: " + count);
            faces = (object[])swMidSurfaceFeature.GetFaces();
            for (i = faces.GetLowerBound(0); i <= faces.GetUpperBound(0); i++)
            {
                swFace = (Face2)faces[i];
                Debug.Print("Area of face " + i + " of midsurface feature: " + swFace.GetArea());
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Areas of MidSurface Faces Example (VBA)

This example shows how to get the areas of mid-surface faces.

```
'-----
' Postconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects two faces and inserts a midsurface feature.
' 3. Gets the number of faces in the midsurface feature.
' 4. Gets the areas of the faces in the midsurface feature.
' 5. Examine the Immediate window, FeatureManager design
'     tree, and graphics area.
'
' NOTE: Because the part used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swMidSurfaceFeature As SldWorks.MidSurface3
Dim swFeature As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFace As SldWorks.Face2
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim count As Long
Dim faces As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", -5.33080255641494E-02, 2.9999999999727E-02, 1.31069871973182E-02, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", -3.70905424398416E-02, 0, 2.89438729892595E-02, True, 0, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    swFeatureManager.InsertMidSurface Nothing, Nothing, 0#, False
    status = swModelDocExt.SelectByID("Surface-MidSurface1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swMidSurfaceFeature = swFeature.GetSpecificFeature2
    count = swMidSurfaceFeature.GetFaceCount
    Debug.Print "Number of faces for midsurface feature: " & count
    faces = swMidSurfaceFeature.GetFaces
    For i = LBound(faces) To UBound(faces)
        Set swFace = faces(i)
        Debug.Print "Area of face " & i & " of midsurface feature: " & swFace.GetArea
    Next i

End Sub
```

Get Areas of MidSurface Faces Example (VB.NET)

This example shows how to get the areas of mid-surface faces.

```

-----
Postconditions:
1. Verify that the part to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the part.
2. Selects two faces and inserts a midsurface feature.
3. Gets the number of faces in the midsurface feature.
4. Gets the areas of the faces in the midsurface feature.
5. Examine the Immediate window, FeatureManager design
   tree, and graphics area.

NOTE: Because the part used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swMidSurfaceFeature As MidSurface3
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swFace As Face2
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim count As Integer
        Dim faces() As Object
        Dim i As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\box.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("", "FACE", -0.0533080255641494, 0.02999999999999727, 0.0131069871973182, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "FACE", -0.0370905424398416, 0, 0.0289438729892595, True, 0, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeatureManager.InsertMidSurface(Nothing, Nothing, 0.0#, False)
        status = swModelDocExt.SelectByID("Surface-MidSurface1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swMidSurfaceFeature = swFeature.GetSpecificFeature2
        count = swMidSurfaceFeature.GetFaceCount
        Debug.Print("Number of faces for midsurface feature: " & count)
        faces = swMidSurfaceFeature.GetFaces
        For i = LBound(faces) To UBound(faces)
            swFace = faces(i)
            Debug.Print("Area of face " & i & " of midsurface feature: " & swFace.GetArea)
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Arrow in Projected View Example (C#)

This example shows how to get the arrow in a projected view.

```
//-----
// Preconditions:
// 1. Open a drawing that has a projected view that contains
//    an arrow with a label.
// 2. Select that projected view.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets information about the projected view's arrow.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetLabelProjectionArrowCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void ProcessTextFormat(SldWorks swApp, ModelDoc2 swModel, TextFormat swTextFormat)
        {
            Debug.Print("    BackWards          = " + swTextFormat.BackWards);
            Debug.Print("    Bold              = " + swTextFormat.Bold);
            Debug.Print("    CharHeight        = " + swTextFormat.CharHeight);
            Debug.Print("    CharHeightInPts   = " + swTextFormat.CharHeightInPts);
            Debug.Print("    CharSpacingFactor = " + swTextFormat.CharSpacingFactor);
            Debug.Print("    Escapement        = " + swTextFormat.Escapement);
            Debug.Print("    IsHeightSpecifiedInPts = " + swTextFormat.IsHeightSpecifiedInPts());
            Debug.Print("    Italic            = " + swTextFormat.Italic);
            Debug.Print("    LineLength        = " + swTextFormat.LineLength);
            Debug.Print("    LineSpacing       = " + swTextFormat.LineSpacing);
            Debug.Print("    ObliqueAngle      = " + swTextFormat.ObliqueAngle);
            Debug.Print("    Strikeout         = " + swTextFormat.Strikeout);
            Debug.Print("    TypeFaceName      = " + swTextFormat.TypeFaceName);
            Debug.Print("    Underline         = " + swTextFormat.Underline);
            Debug.Print("    UpsideDown        = " + swTextFormat.UpsideDown);
            Debug.Print("    Vertical          = " + swTextFormat.Vertical);
            Debug.Print("    WidthFactor       = " + swTextFormat.WidthFactor);
            Debug.Print("");
        }

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DrawingDoc swDraw = default(DrawingDoc);
            View swView = default(View);
            ProjectionArrow swProjArr = default(ProjectionArrow);
            View swBaseView = default(View);
            double[] vCoord = null;
            TextFormat swTextFormat = default(TextFormat);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            swProjArr = (ProjectionArrow)swView.GetProjectionArrow();
            swBaseView = (View)swProjArr.GetView();
            swTextFormat = (TextFormat)swProjArr.GetTextFormat();
            vCoord = (double[])swProjArr.GetCoordinates();
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swView.Name + " --> " + swBaseView.Name);
            Debug.Print(" Coordinates = (" + vCoord[0] * 1000.0 + ", " + vCoord[1] * 1000.0 + ", " + vCoord[2] * 1000.0 + ") mm");
            Debug.Print(" Label = " + swProjArr.GetLabel());
            Debug.Print(" Use document text format = " + swProjArr.GetUseDocTextFormat());

            ProcessTextFormat(swApp, swModel, swTextFormat);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Arrow in Projected View Example (VBA)

This example shows how to get the arrow in a projected view.

```

'-----
' Preconditions:
' 1. Open a drawing that has a projected view that contains
'    an arrow with a label.
' 2. Select that projected view.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets information about the projected view's arrow.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub ProcessTextFormat(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTextFormat As SldWorks.TextFormat)

    Debug.Print "    BackWards          = " & swTextFormat.BackWards
    Debug.Print "    Bold              = " & swTextFormat.Bold
    Debug.Print "    CharHeight         = " & swTextFormat.CharHeight
    Debug.Print "    CharHeightInPts    = " & swTextFormat.CharHeightInPts
    Debug.Print "    CharSpacingFactor  = " & swTextFormat.CharSpacingFactor
    Debug.Print "    Escapement         = " & swTextFormat.Escapement
    Debug.Print "    IsHeightSpecifiedInPts = " & swTextFormat.IsHeightSpecifiedInPts
    Debug.Print "    Italic             = " & swTextFormat.Italic
    Debug.Print "    LineLength         = " & swTextFormat.LineLength
    Debug.Print "    LineSpacing        = " & swTextFormat.LineSpacing
    Debug.Print "    ObliqueAngle       = " & swTextFormat.ObliqueAngle
    Debug.Print "    Strikeout          = " & swTextFormat.Strikeout
    Debug.Print "    TypeFaceName       = " & swTextFormat.TypeFaceName
    Debug.Print "    Underline          = " & swTextFormat.Underline
    Debug.Print "    UpsideDown         = " & swTextFormat.UpsideDown
    Debug.Print "    Vertical           = " & swTextFormat.Vertical
    Debug.Print "    WidthFactor        = " & swTextFormat.WidthFactor

    Debug.Print ""
End Sub

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swDraw              As SldWorks.DrawingDoc
    Dim swView              As SldWorks.View
    Dim swProjArr            As SldWorks.ProjectionArrow
    Dim swBaseView          As SldWorks.View
    Dim vCoord              As Variant
    Dim swTextFormat        As SldWorks.TextFormat
    Dim bRet                As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Set swProjArr = swView.GetProjectionArrow
    Set swBaseView = swProjArr.GetView
    Set swTextFormat = swProjArr.GetTextFormat

    vCoord = swProjArr.GetCoordinates

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "    " & swView.Name & " --> " & swBaseView.Name
    Debug.Print "    Coordinates          = (" & vCoord(0) * 1000# & ", " & vCoord(1) * 1000# & ", " & vCoord(2) * 1000# & ") mm"
    Debug.Print "    Label                = " & swProjArr.GetLabel
    Debug.Print "    Use document text format = " & swProjArr.GetUseDocTextFormat

    ProcessTextFormat swApp, swModel, swTextFormat

End Sub

```

Get Arrow in Projected View Example (VB.NET)

This example shows how to get the arrow in a projected view.

```

-----
Preconditions:
1. Open a drawing that has a projected view that contains
   an arrow with a label.
2. Select that projected view.
3. Open the Immediate window.
Postconditions:
1. Gets information about the projected view's arrow.
2. Examine the Immediate window.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Sub ProcessTextFormat(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swTextFormat As TextFormat)
        Debug.Print(" BackWards = " & swTextFormat.Backwards)
        Debug.Print(" Bold = " & swTextFormat.Bold)
        Debug.Print(" CharHeight = " & swTextFormat.CharHeight)
        Debug.Print(" CharHeightInPts = " & swTextFormat.CharHeightInPts)
        Debug.Print(" CharSpacingFactor = " & swTextFormat.CharSpacingFactor)
        Debug.Print(" Escapement = " & swTextFormat.Escapement)
        Debug.Print(" IsHeightSpecifiedInPts = " & swTextFormat.IsHeightSpecifiedInPts)
        Debug.Print(" Italic = " & swTextFormat.Italic)
        Debug.Print(" LineLength = " & swTextFormat.LineLength)
        Debug.Print(" LineSpacing = " & swTextFormat.LineSpacing)
        Debug.Print(" ObliqueAngle = " & swTextFormat.ObliqueAngle)
        Debug.Print(" Strikeout = " & swTextFormat.Strikeout)
        Debug.Print(" TypeFaceName = " & swTextFormat.TypeFaceName)
        Debug.Print(" Underline = " & swTextFormat.Underline)
        Debug.Print(" UpsideDown = " & swTextFormat.UpsideDown)
        Debug.Print(" Vertical = " & swTextFormat.Vertical)
        Debug.Print(" WidthFactor = " & swTextFormat.WidthFactor)
        Debug.Print("")
    End Sub

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swDraw As DrawingDoc
        Dim swView As View
        Dim swProjArr As ProjectionArrow
        Dim swBaseView As View
        Dim vCoord As Object
        Dim swTextFormat As TextFormat
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swDraw = swModel
        swSelMgr = swModel.SelectionManager
        swView = swSelMgr.GetSelectedObject6(1, -1)
        swProjArr = swView.GetProjectionArrow
        swBaseView = swProjArr.GetView
        swTextFormat = swProjArr.GetTextFormat
        vCoord = swProjArr.GetCoordinates
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swView.Name & " --> " & swBaseView.Name)
        Debug.Print(" Coordinates = (" & vCoord(0) * 1000.0# & ", " & vCoord(1) * 1000.0# & ", " & vCoord(2) * 1000.0# & ") mm")
        Debug.Print(" Label = " & swProjArr.GetLabel)
        Debug.Print(" Use document text format = " & swProjArr.GetUseDocTextFormat)

        ProcessTextFormat(swApp, swModel, swTextFormat)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

```
End Class
```

Get Assembly Bounding Box Using Assembly Example (C#)

This example shows how to get the box bounding an assembly using the assembly.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Adds a 3D sketch to the assembly showing the bounding box.
// 2. Examine the graphics area and Immediate window to verify.
//
// NOTE: The assembly bounding box is approximated
// and is oriented with the assembly coordinate system.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetBoxAssemblyCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            object Box = null;
            double[] BoxArray = new double[6];
            double X_max = 0;
            double X_min = 0;
            double Y_max = 0;
            double Y_min = 0;
            double Z_max = 0;
            double Z_min = 0;
            SketchManager swSketchMgr = default(SketchManager);
            SketchPoint[] swSketchPt = new SketchPoint[9];
            SketchSegment[] swSketchSeg = new SketchSegment[13];

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;

            Box = (object)swAssy.GetBox((int)swBoundingBoxOptions_e.swBoundingBoxIncludeRefPlanes);
            BoxArray = (double[])Box;

            // Initialize values
            X_max = BoxArray[3];
            X_min = BoxArray[0];
            Y_max = BoxArray[4];
            Y_min = BoxArray[1];
            Z_max = BoxArray[5];
            Z_min = BoxArray[2];

            swSketchMgr = (SketchManager)swModel.SketchManager;

            swSketchMgr.Insert3DSketch(true);
            swSketchMgr.AddToDB = true;

            // Draw points at each corner of bounding box
            swSketchPt[0] = swSketchMgr.CreatePoint(X_min, Y_min, Z_min);
            swSketchPt[1] = swSketchMgr.CreatePoint(X_min, Y_min, Z_max);
            swSketchPt[2] = swSketchMgr.CreatePoint(X_min, Y_max, Z_min);
            swSketchPt[3] = swSketchMgr.CreatePoint(X_min, Y_max, Z_max);
            swSketchPt[4] = swSketchMgr.CreatePoint(X_max, Y_min, Z_min);
            swSketchPt[5] = swSketchMgr.CreatePoint(X_max, Y_min, Z_max);
            swSketchPt[6] = swSketchMgr.CreatePoint(X_max, Y_max, Z_min);
            swSketchPt[7] = swSketchMgr.CreatePoint(X_max, Y_max, Z_max);

            // Draw bounding box
            swSketchSeg[0] = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_max, Y_min, Z_min);
            swSketchSeg[1] = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_min, Z_max);
            swSketchSeg[2] = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_min, Y_min, Z_max);
            swSketchSeg[3] = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_min, Z_min);
            swSketchSeg[4] = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_min, Y_max, Z_min);
            swSketchSeg[5] = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_max, Z_max);
            swSketchSeg[6] = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min);
            swSketchSeg[7] = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_max, Z_min);
            swSketchSeg[8] = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min);
            swSketchSeg[9] = swSketchMgr.CreateLine(X_max, Y_max, Z_min, X_max, Y_max, Z_max);
            swSketchSeg[10] = swSketchMgr.CreateLine(X_max, Y_max, Z_max, X_min, Y_max, Z_max);
            swSketchSeg[11] = swSketchMgr.CreateLine(X_min, Y_max, Z_max, X_min, Y_max, Z_min);

            swSketchMgr.AddToDB = false;
            swSketchMgr.Insert3DSketch(true);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Assembly Bounding Box using Assembly Example (VBA)

This example shows how to get the box bounding an assembly using the assembly.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Adds a 3D sketch to the assembly showing the bounding box.
' 2. Examine the graphics area and Immediate window to verify.
'
' NOTE: The bounding box is approximated and oriented
' with the assembly coordinate system.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim vBox As Variant
Dim X_max As Double
Dim X_min As Double
Dim Y_max As Double
Dim Y_min As Double
Dim Z_max As Double
Dim Z_min As Double
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchPt(8) As SldWorks.SketchPoint
Dim swSketchSeg(12) As SldWorks.SketchSegment

Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swAssy = swModel

vBox = swAssy.GetBox(swBoundingBoxIncludeRefPlanes)

' Initialize values
X_max = vBox(3)
X_min = vBox(0)
Y_max = vBox(4)
Y_min = vBox(1)
Z_max = vBox(5)
Z_min = vBox(2)

Debug.Print "Assembly Bounding Box (" + swModel.GetPathName + ") = "
Debug.Print " (" + Str(X_min * 1000#) + ", " + Str(Y_min * 1000#) + ", " + Str(Z_min * 1000#) + ") mm"
Debug.Print " (" + Str(X_max * 1000#) + ", " + Str(Y_max * 1000#) + ", " + Str(Z_max * 1000#) + ") mm"

Set swSketchMgr = swModel.SketchManager
swSketchMgr.Insert3DSketch True
swSketchMgr.AddToDB = True

' Draw points at each corner of bounding box
Set swSketchPt(0) = swSketchMgr.CreatePoint(X_min, Y_min, Z_min)
Set swSketchPt(1) = swSketchMgr.CreatePoint(X_min, Y_min, Z_max)
Set swSketchPt(2) = swSketchMgr.CreatePoint(X_min, Y_max, Z_min)
Set swSketchPt(3) = swSketchMgr.CreatePoint(X_min, Y_max, Z_max)
Set swSketchPt(4) = swSketchMgr.CreatePoint(X_max, Y_min, Z_min)
Set swSketchPt(5) = swSketchMgr.CreatePoint(X_max, Y_min, Z_max)
Set swSketchPt(6) = swSketchMgr.CreatePoint(X_max, Y_max, Z_min)
Set swSketchPt(7) = swSketchMgr.CreatePoint(X_max, Y_max, Z_max)

' Draw bounding box
Set swSketchSeg(0) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_max, Y_min, Z_min)
Set swSketchSeg(1) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_min, Z_max)
Set swSketchSeg(2) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_min, Y_min, Z_max)
Set swSketchSeg(3) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_min, Z_min)
Set swSketchSeg(4) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_min, Y_max, Z_min)
Set swSketchSeg(5) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_max, Z_max)
Set swSketchSeg(6) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_max, Z_min)
Set swSketchSeg(7) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_max, Y_max, Z_max)
Set swSketchSeg(8) = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min)
Set swSketchSeg(9) = swSketchMgr.CreateLine(X_max, Y_max, Z_min, X_max, Y_max, Z_max)
Set swSketchSeg(10) = swSketchMgr.CreateLine(X_max, Y_max, Z_max, X_min, Y_max, Z_max)
Set swSketchSeg(11) = swSketchMgr.CreateLine(X_min, Y_max, Z_max, X_min, Y_max, Z_min)

swSketchMgr.AddToDB = False
swSketchMgr.Insert3DSketch True
```

End Sub

Get Assembly Bounding Box Using Assembly Example (VB.NET)

This example shows how to get the box bounding an assembly using the assembly.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Adds a 3D sketch to the assembly showing the bounding box.
' 2. Examine the graphics area and Immediate window to verify.
'
' NOTE: The bounding box for the assembly is approximated
' and is oriented with the assembly coordinate system.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim vBox As Object
        Dim X_max As Double
        Dim X_min As Double
        Dim Y_max As Double
        Dim Y_min As Double
        Dim Z_max As Double
        Dim Z_min As Double
        Dim swSketchMgr As SketchManager
        Dim swSketchPt(8) As SketchPoint
        Dim swSketchSeg(12) As SketchSegment

        swModel = swApp.ActiveDoc
        swAssy = swModel

        vBox = swAssy.GetBox(swBoundingBoxOptions_e.swBoundingBoxIncludeRefPlanes)

        ' Initialize values
        X_max = vBox(3)
        X_min = vBox(0)
        Y_max = vBox(4)
        Y_min = vBox(1)
        Z_max = vBox(5)
        Z_min = vBox(2)

        Debug.Print("Assembly Bounding Box (" + swModel.GetPathName + ") = ")
        Debug.Print(" (" + Str(X_min * 1000.0#) + "," + Str(Y_min * 1000.0#) + "," + Str(Z_min * 1000.0#) + ") mm")
        Debug.Print(" (" + Str(X_max * 1000.0#) + "," + Str(Y_max * 1000.0#) + "," + Str(Z_max * 1000.0#) + ") mm")
        swSketchMgr = swModel.SketchManager

        swSketchMgr.Insert3DSketch(True)
        swSketchMgr.AddToDB = True

        ' Draw points at each corner of bounding box
        swSketchPt(0) = swSketchMgr.CreatePoint(X_min, Y_min, Z_min)
        swSketchPt(1) = swSketchMgr.CreatePoint(X_min, Y_min, Z_max)
        swSketchPt(2) = swSketchMgr.CreatePoint(X_min, Y_max, Z_min)
        swSketchPt(3) = swSketchMgr.CreatePoint(X_min, Y_max, Z_max)
        swSketchPt(4) = swSketchMgr.CreatePoint(X_max, Y_min, Z_min)
        swSketchPt(5) = swSketchMgr.CreatePoint(X_max, Y_min, Z_max)
        swSketchPt(6) = swSketchMgr.CreatePoint(X_max, Y_max, Z_min)
        swSketchPt(7) = swSketchMgr.CreatePoint(X_max, Y_max, Z_max)

        ' Draw bounding box
        swSketchSeg(0) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_max, Y_min, Z_min)
        swSketchSeg(1) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_min, Z_max)
        swSketchSeg(2) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_min, Y_min, Z_max)
        swSketchSeg(3) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_min, Z_min)
        swSketchSeg(4) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_min, Y_max, Z_min)
        swSketchSeg(5) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_max, Z_max)
        swSketchSeg(6) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_max, Z_min)
        swSketchSeg(7) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_max, Y_max, Z_max)
        swSketchSeg(8) = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min)
        swSketchSeg(9) = swSketchMgr.CreateLine(X_max, Y_max, Z_min, X_max, Y_max, Z_max)
        swSketchSeg(10) = swSketchMgr.CreateLine(X_max, Y_max, Z_max, X_min, Y_max, Z_max)
        swSketchSeg(11) = swSketchMgr.CreateLine(X_min, Y_max, Z_max, X_min, Y_max, Z_min)

        swSketchMgr.AddToDB = False
        swSketchMgr.Insert3DSketch(True)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Get Assembly Bounding Box Using Components Example (C#)

This example shows how to get a bounding box for an assembly.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Ensure that all components in the assembly are fully resolved.
// 3. Open the Immediate window.
// 4. Run the macro.
//
// Postconditions:
// 1. Adds a 3D sketch to the assembly showing the bounding box.
// 2. Examine the graphics area and Immediate window to verify.
//
// NOTES:
// * Because all assembly component bounding boxes are
//   approximated, the bounding box for the assembly
//   is also approximated.
// * Resulting bounding box is oriented with the assembly
//   coordinate system.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.sldworks;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetBoxComponents2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public double GetMax(double Val1, double Val2, double Val3)
        {
            double functionReturnValue = 0;
            // Finds maximum of 3 values
            functionReturnValue = Val1;
            if (Val2 > functionReturnValue)
            {
                functionReturnValue = Val2;
            }
            if (Val3 > functionReturnValue)
            {
                functionReturnValue = Val3;
            }
            return functionReturnValue;
        }

        public double GetMin(double Val1, double Val2, double Val3)
        {
            double functionReturnValue = 0;
            // Finds minimum of 3 values
            functionReturnValue = Val1;
            if (Val2 < functionReturnValue)
            {
                functionReturnValue = Val2;
            }
            if (Val3 < functionReturnValue)
            {
                functionReturnValue = Val3;
            }
            return functionReturnValue;
        }

        public void Main()
        {
            const double MaxDouble = 1.79769313486231E+308;
            const double MinDouble = -1.79769313486231E+308;
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            Configuration swConfig = default(Configuration);
            ConfigurationManager swConfigurationMgr = default(ConfigurationManager);
            Component2 swRootComp = default(Component2);
            object[] vChild = null;
            Component2 swChildComp = default(Component2);
            object Box = null;
            double[] BoxArray = new double[6];
            double X_max = 0;
            double X_min = 0;
            double Y_max = 0;
            double Y_min = 0;
            double Z_max = 0;
            double Z_min = 0;
            SketchManager swSketchMgr = default(SketchManager);
            SketchPoint[] swSketchPt = new SketchPoint[9];
            SketchSegment[] swSketchSeg = new SketchSegment[13];
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swConfigurationMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfig = (Configuration)swConfigurationMgr.ActiveConfiguration;
            swRootComp = (Component2)swConfig.GetRootComponent3(true);

            // Initialize values
            X_max = MinDouble;
            X_min = MaxDouble;
            Y_max = MinDouble;
            Y_min = MaxDouble;
            Z_max = MinDouble;
            Z_min = MaxDouble;

            vChild = (object[])swRootComp.GetChildren();
            for (i = 0; i <= vChild.GetUpperBound(0); i++)
            {
                swChildComp = (Component2)vChild[i];
                if (swChildComp.Visible == (int)swComponentVisibilityState_e.swComponentVisible)

```

```

    {
        Box = (object)swChildComp.GetBox(false, false);
        BoxArray = (double[])Box;
        X_max = GetMax(BoxArray[0], BoxArray[3], X_max);
        X_min = GetMin(BoxArray[0], BoxArray[3], X_min);
        Y_max = GetMax(BoxArray[1], BoxArray[4], Y_max);
        Y_min = GetMin(BoxArray[1], BoxArray[4], Y_min);
        Z_max = GetMax(BoxArray[2], BoxArray[5], Z_max);
        Z_min = GetMin(BoxArray[2], BoxArray[5], Z_min);
    }
}

Debug.Print("Assembly Bounding Box (" + swModel.GetPathName() + ") = ");
Debug.Print(" (" + (X_min * 1000.0) + ", " + (Y_min * 1000.0) + ", " + (Z_min * 1000.0) + ") mm");
Debug.Print(" (" + (X_max * 1000.0) + ", " + (Y_max * 1000.0) + ", " + (Z_max * 1000.0) + ") mm");

swSketchMgr = swModel.SketchManager;

swSketchMgr.Insert3DSketch(true);
swSketchMgr.AddToDB = true;

// Draw points at each corner of bounding box
swSketchPt[0] = swSketchMgr.CreatePoint(X_min, Y_min, Z_min);
swSketchPt[1] = swSketchMgr.CreatePoint(X_min, Y_min, Z_max);
swSketchPt[2] = swSketchMgr.CreatePoint(X_min, Y_max, Z_min);
swSketchPt[3] = swSketchMgr.CreatePoint(X_min, Y_max, Z_max);
swSketchPt[4] = swSketchMgr.CreatePoint(X_max, Y_min, Z_min);
swSketchPt[5] = swSketchMgr.CreatePoint(X_max, Y_min, Z_max);
swSketchPt[6] = swSketchMgr.CreatePoint(X_max, Y_max, Z_min);
swSketchPt[7] = swSketchMgr.CreatePoint(X_max, Y_max, Z_max);

// Draw bounding box
swSketchSeg[0] = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_max, Y_min, Z_min);
swSketchSeg[1] = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_min, Z_max);
swSketchSeg[2] = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_min, Y_min, Z_max);
swSketchSeg[3] = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_min, Z_min);
swSketchSeg[4] = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_min, Y_max, Z_min);
swSketchSeg[5] = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_max, Z_max);
swSketchSeg[6] = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_max, Z_min);
swSketchSeg[7] = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_max, Y_max, Z_max);
swSketchSeg[8] = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min);
swSketchSeg[9] = swSketchMgr.CreateLine(X_max, Y_max, Z_min, X_max, Y_max, Z_max);
swSketchSeg[10] = swSketchMgr.CreateLine(X_max, Y_max, Z_max, X_min, Y_max, Z_max);
swSketchSeg[11] = swSketchMgr.CreateLine(X_min, Y_max, Z_max, X_min, Y_max, Z_min);

swSketchMgr.AddToDB = false;
swSketchMgr.Insert3DSketch(true);
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Assembly Bounding Box Using Components Example (VBA)

This example shows how to get a bounding box for an assembly.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Ensure that all components in the assembly are fully resolved.
' 3. Open the Immediate window.
' 4. Run the macro.
'
' Postconditions:
' 1. Adds a 3D sketch to the assembly showing the bounding box.
' 2. Examine the graphics area and Immediate window to verify.
'
' NOTES:
' * Because all assembly component bounding boxes are
'   approximated, the bounding box for the assembly
'   is also approximated.
' * Resulting bounding box is oriented with the assembly
'   coordinate system.
'-----
```

Option Explicit

Function GetMax(ByVal Val1 As Double, ByVal Val2 As Double, ByVal Val3 As Double) As Double

' Finds maximum of 3 values

```
    GetMax = Val1
    If Val2 > GetMax Then
        GetMax = Val2
    End If
    If Val3 > GetMax Then
        GetMax = Val3
    End If
```

End Function

Function GetMin(ByVal Val1 As Double, ByVal Val2 As Double, ByVal Val3 As Double) As Double

' Finds minimum of 3 values

```
    GetMin = Val1
    If Val2 < GetMin Then
        GetMin = Val2
    End If
    If Val3 < GetMin Then
        GetMin = Val3
    End If
```

End Function

Sub main()

```
    Const MaxDouble As Double = 1.79769313486231E+308
    Const MinDouble As Double = -1.79769313486231E+308
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swConfigurationMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim vChild As Variant
    Dim swChildComp As SldWorks.Component2
    Dim vBox As Variant
    Dim X_max As Double
    Dim X_min As Double
    Dim Y_max As Double
    Dim Y_min As Double
    Dim Z_max As Double
    Dim Z_min As Double
    Dim swSketchMgr As SldWorks.SketchManager
    Dim swSketchPt(8) As SldWorks.SketchPoint
    Dim swSketchSeg(12) As SldWorks.SketchSegment
    Dim i As Long
```

```
Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swAssy = swModel
Set swConfigurationMgr = swModel.ConfigurationManager
Set swConfig = swConfigurationMgr.ActiveConfiguration
Set swRootComp = swConfig.GetRootComponent3(True)
```

' Initialize values

```
X_max = MinDouble
X_min = MaxDouble
Y_max = MinDouble
Y_min = MaxDouble
Z_max = MinDouble
Z_min = MaxDouble
```

vChild = swRootComp.GetChildren

For i = 0 To UBound(vChild)

Set swChildComp = vChild(i)

If swChildComp.Visible = swComponentVisible Then

vBox = swChildComp.GetBox(False, False)

X_max = GetMax(vBox(0), vBox(3), X_max)

X_min = GetMin(vBox(0), vBox(3), X_min)

```

        Y_max = GetMax(vBox(1), vBox(4), Y_max)
        Y_min = GetMin(vBox(1), vBox(4), Y_min)
        Z_max = GetMax(vBox(2), vBox(5), Z_max)
        Z_min = GetMin(vBox(2), vBox(5), Z_min)
    End If
Next i

Debug.Print "Assembly Bounding Box (" + swModel.GetPathName + ") = "
Debug.Print " (" + Str(X_min * 1000#) + "," + Str(Y_min * 1000#) + "," + Str(Z_min * 1000#) + ") mm"
Debug.Print " (" + Str(X_max * 1000#) + "," + Str(Y_max * 1000#) + "," + Str(Z_max * 1000#) + ") mm"

Set swSketchMgr = swModel.SketchManager

swSketchMgr.Insert3DSketch True
swSketchMgr.AddToDB = True

' Draw points at each corner of bounding box
Set swSketchPt(0) = swSketchMgr.CreatePoint(X_min, Y_min, Z_min)
Set swSketchPt(1) = swSketchMgr.CreatePoint(X_min, Y_min, Z_max)
Set swSketchPt(2) = swSketchMgr.CreatePoint(X_min, Y_max, Z_min)
Set swSketchPt(3) = swSketchMgr.CreatePoint(X_min, Y_max, Z_max)
Set swSketchPt(4) = swSketchMgr.CreatePoint(X_max, Y_min, Z_min)
Set swSketchPt(5) = swSketchMgr.CreatePoint(X_max, Y_min, Z_max)
Set swSketchPt(6) = swSketchMgr.CreatePoint(X_max, Y_max, Z_min)
Set swSketchPt(7) = swSketchMgr.CreatePoint(X_max, Y_max, Z_max)

' Draw bounding box
Set swSketchSeg(0) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_max, Y_min, Z_min)
Set swSketchSeg(1) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_min, Z_max)
Set swSketchSeg(2) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_min, Y_min, Z_max)
Set swSketchSeg(3) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_min, Z_min)
Set swSketchSeg(4) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_min, Y_max, Z_min)
Set swSketchSeg(5) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_max, Z_max)
Set swSketchSeg(6) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_max, Z_min)
Set swSketchSeg(7) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_max, Y_max, Z_max)
Set swSketchSeg(8) = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min)
Set swSketchSeg(9) = swSketchMgr.CreateLine(X_max, Y_max, Z_min, X_max, Y_max, Z_max)
Set swSketchSeg(10) = swSketchMgr.CreateLine(X_max, Y_max, Z_max, X_min, Y_max, Z_max)
Set swSketchSeg(11) = swSketchMgr.CreateLine(X_min, Y_max, Z_max, X_min, Y_max, Z_min)

swSketchMgr.AddToDB = False
swSketchMgr.Insert3DSketch True

End Sub

```

Get Assembly Bounding Box Using Components Example (VB.NET)

This example shows how to get a bounding box for an assembly.

```

-----
Preconditions:
1. Open an assembly document.
2. Ensure that all components in the assembly are fully resolved.
3. Open the Immediate window.
4. Run the macro.

Postconditions:
1. Adds a 3D sketch to the assembly showing the bounding box.
2. Examine the graphics area and Immediate window to verify.

NOTES:
* Because all assembly component bounding boxes are
  approximated, the bounding box for the assembly
  is also approximated.
* Resulting bounding box is oriented with the assembly
  coordinate system.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Function GetMax(ByVal Val1 As Double, ByVal Val2 As Double, ByVal Val3 As Double) As Double
        ' Finds maximum of 3 values
        GetMax = Val1
        If Val2 > GetMax Then
            GetMax = Val2
        End If
        If Val3 > GetMax Then
            GetMax = Val3
        End If
    End Function

    Function GetMin(ByVal Val1 As Double, ByVal Val2 As Double, ByVal Val3 As Double) As Double
        ' Finds minimum of 3 values
        GetMin = Val1
        If Val2 < GetMin Then
            GetMin = Val2
        End If
        If Val3 < GetMin Then
            GetMin = Val3
        End If
    End Function

    Public Sub Main()

        Const MaxDouble As Double = 1.79769313486231E+308
        Const MinDouble As Double = -1.79769313486231E+308
        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim swConfigurationMgr As ConfigurationManager
        Dim swConfig As Configuration
        Dim swRootComp As Component2
        Dim vChild As Object
        Dim swChildComp As Component2
        Dim vBox As Object
        Dim X_max As Double
        Dim X_min As Double
        Dim Y_max As Double
        Dim Y_min As Double
        Dim Z_max As Double
        Dim Z_min As Double
        Dim swSketchMgr As SketchManager
        Dim swSketchPt(8) As SketchPoint
        Dim swSketchSeg(12) As SketchSegment
        Dim i As Integer

        swApp = CreateObject("SldWorks.Application")
        swModel = swApp.ActiveDoc
        swAssy = swModel
        swConfigurationMgr = swModel.ConfigurationManager
        swConfig = swConfigurationMgr.ActiveConfiguration
        swRootComp = swConfig.GetRootComponent3(True)

        ' Initialize values
        X_max = MinDouble
        X_min = MaxDouble
        Y_max = MinDouble
        Y_min = MaxDouble
        Z_max = MinDouble
        Z_min = MaxDouble

        vChild = swRootComp.GetChildren
        For i = 0 To UBound(vChild)
            swChildComp = vChild(i)
            If swChildComp.Visible = swComponentVisibilityState_e.swComponentVisible Then
                vBox = swChildComp.GetBox(False, False)
                X_max = GetMax(vBox(0), vBox(3), X_max)
                X_min = GetMin(vBox(0), vBox(3), X_min)
                Y_max = GetMax(vBox(1), vBox(4), Y_max)
                Y_min = GetMin(vBox(1), vBox(4), Y_min)
                Z_max = GetMax(vBox(2), vBox(5), Z_max)
                Z_min = GetMin(vBox(2), vBox(5), Z_min)
            End If
        Next i

        Debug.Print("Assembly Bounding Box (" & swModel.GetPathName & ") = ")
        Debug.Print("(" & Str(X_min * 1000.0#) & " , " & Str(Y_min * 1000.0#) & " , " & Str(Z_min * 1000.0#) & ") mm")
        Debug.Print("(" & Str(X_max * 1000.0#) & " , " & Str(Y_max * 1000.0#) & " , " & Str(Z_max * 1000.0#) & ") mm")
    End Sub
End Class

```

```
swSketchMgr = swModel.SketchManager
```

```
swSketchMgr.Insert3DSketch(True)  
swSketchMgr.AddToDB = True
```

```
' Draw points at each corner of bounding box
```

```
swSketchPt(0) = swSketchMgr.CreatePoint(X_min, Y_min, Z_min)  
swSketchPt(1) = swSketchMgr.CreatePoint(X_min, Y_min, Z_max)  
swSketchPt(2) = swSketchMgr.CreatePoint(X_min, Y_max, Z_min)  
swSketchPt(3) = swSketchMgr.CreatePoint(X_min, Y_max, Z_max)  
swSketchPt(4) = swSketchMgr.CreatePoint(X_max, Y_min, Z_min)  
swSketchPt(5) = swSketchMgr.CreatePoint(X_max, Y_min, Z_max)  
swSketchPt(6) = swSketchMgr.CreatePoint(X_max, Y_max, Z_min)  
swSketchPt(7) = swSketchMgr.CreatePoint(X_max, Y_max, Z_max)
```

```
' Draw bounding box
```

```
swSketchSeg(0) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_max, Y_min, Z_min)  
swSketchSeg(1) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_min, Z_max)  
swSketchSeg(2) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_min, Y_min, Z_max)  
swSketchSeg(3) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_min, Z_min)  
swSketchSeg(4) = swSketchMgr.CreateLine(X_min, Y_min, Z_min, X_min, Y_max, Z_min)  
swSketchSeg(5) = swSketchMgr.CreateLine(X_min, Y_min, Z_max, X_min, Y_max, Z_max)  
swSketchSeg(6) = swSketchMgr.CreateLine(X_max, Y_min, Z_min, X_max, Y_max, Z_min)  
swSketchSeg(7) = swSketchMgr.CreateLine(X_max, Y_min, Z_max, X_max, Y_max, Z_max)  
swSketchSeg(8) = swSketchMgr.CreateLine(X_min, Y_max, Z_min, X_max, Y_max, Z_min)  
swSketchSeg(9) = swSketchMgr.CreateLine(X_max, Y_max, Z_min, X_max, Y_max, Z_max)  
swSketchSeg(10) = swSketchMgr.CreateLine(X_max, Y_max, Z_max, X_min, Y_max, Z_max)  
swSketchSeg(11) = swSketchMgr.CreateLine(X_min, Y_max, Z_max, X_min, Y_max, Z_min)
```

```
swSketchMgr.AddToDB = False  
swSketchMgr.Insert3DSketch(True)
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Get Available Custom Properties for BOM Table Example (VBA)

This example shows how to get a list of all of the custom properties available to a BOM table.

```
'-----
' Preconditions:
' 1. Specified drawing and BOM table template exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Inserts an indented BOM table.
' 3. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save any changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swBOMAnnotation As SldWorks.BomTableAnnotation
Dim status As Boolean
Dim anchorType As Long
Dim bomType As Long
Dim errors As Long
Dim warnings As Long
Dim tableTemplate As String
Dim customPropArray As Variant
Dim config As String
Dim customProp As Variant

Option Explicit

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw", swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension
    status = swDrawing.ActivateView("Drawing View1")
    Set swView = swDrawing.ActiveDrawingView

    ' Insert indented BOM table and get available custom properties
    anchorType = swBOMConfigurationAnchor_TopLeft
    bomType = swBomType_Indented
    tableTemplate = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt"
    config = ""
    Set swBOMAnnotation = swView.InsertBomTable4(False, 0.4, 0.3, anchorType, bomType, config, tableTemplate, False, swNumberingType_Detailed, True)
    customPropArray = swBOMAnnotation.GetAllCustomProperties
    For Each customProp In customPropArray
        Debug.Print " " & customProp
    Next customProp

    swDrawing.ForceRebuild

End Sub
```

Get Axis of Revolve Feature Example (VBA)

This example shows how to get the type of axis of revolution and the axis of revolution for a revolve feature.

```
'-----  
'  
' Preconditions:  
' (1) Part document is open.  
' (2) Revolve feature exists and is selected.  
'  
' Postconditions: The axis of revolution is identified and  
selected, and then deselected.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim featdata As SldWorks.RevolveFeatureData2  
Dim feat As SldWorks.Feature  
Dim skobj As SldWorks.SketchSegment  
Dim edgeobj As SldWorks.entity  
Dim axisobj As SldWorks.refAxis  
Dim boolstatus As Boolean  
Dim longstatus As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set feat = swSelMgr.GetSelectedObject5(1)  
Set featdata = feat.GetDefinition  
  
boolstatus = featdata.AccessSelections(swModel, Nothing)  
longstatus = featdata.GetAxisType  
Set skobj = featdata.axis  
  
Select Case longstatus  
Case SwConst.swSelSKETCHSEGS  
Set skobj = featdata.axis  
boolstatus = skobj.Select4(False, Nothing)  
Case SwConst.swSelDATUMAXES  
Set axisobj = featdata.axis  
Set feat = axisobj  
boolstatus = feat.Select2(False, 0)  
Case SwConst.swSelEDGES  
Set edgeobj = featdata.axis  
boolstatus = edgeobj.Select4(False, Nothing)  
End Select  
  
featdata.ReleaseSelectionAccess  
End Sub
```

Get B-Spline Surface Parameterization Data Example (C#)

This example shows how to get B-spline parameterization data for a selected face or surface.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
// 2. Select a face.
// 3. Open the Immediate Window.
//
// Postconditions: Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace MacroBSurf_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Face2 swFace = default(Face2);
            Surface swSurf = default(Surface);
            SurfaceParameterizationData swSurfParam = default(SurfaceParameterizationData);
            BSurfParamData swBSurfParam = default(BSurfParamData);
            bool sense = false;
            object UKnots = null;
            object VKnots = null;
            object vCtrlPts = null;
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject5(1);
            swSurf = (Surface)swFace.GetSurface();
            swSurfParam = (SurfaceParameterizationData)swSurf.Parameterization2();
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Surface:");
            Debug.Print(" U minimum is: " + swSurfParam.UMin);
            Debug.Print(" U minimum bound type is: " + swSurfParam.UMinBoundType);
            Debug.Print(" U maximum is: " + swSurfParam.UMax);
            Debug.Print(" U maximum bound type is: " + swSurfParam.UMaxBoundType);
            Debug.Print(" U property count is: " + swSurfParam.UPropertyNumber);
            object varUProperties = null;
            int[] uProperties;
            varUProperties = swSurfParam.UProperties;
            uProperties = (int[])varUProperties;
            Debug.Print(" U properties: ");
            for (i = 0; i <= uProperties.GetUpperBound(0); i++)
            {
                Debug.Print(" " + uProperties[i]);
            }

            Debug.Print(" V minimum is: " + swSurfParam.VMin);
            Debug.Print(" V minimum bound type is: " + swSurfParam.VMinBoundType);
            Debug.Print(" V maximum is: " + swSurfParam.VMax);
            Debug.Print(" V maximum bound type is: " + swSurfParam.VMaxBoundType);
            Debug.Print(" V property count is: " + swSurfParam.VPropertyNumber);
            object varVProperties = null;
            int[] vProperties;
            varVProperties = swSurfParam.VProperties;
            vProperties = (int[])varVProperties;
            Debug.Print(" V properties: ");
            for (i = 0; i <= vProperties.GetUpperBound(0); i++)
            {
                Debug.Print(" " + vProperties[i]);
            }

            swBSurfParam = (BSurfParamData)swSurf.GetBSurfParams3(false, false, swSurfParam, 0.01, out sense);

            Debug.Print("U order is: " + swBSurfParam.UOrder);
            Debug.Print("V order is: " + swBSurfParam.VOrder);
            Debug.Print(" Control point column count is: " + swBSurfParam.ControlPointColumnCount);
            Debug.Print(" Control point row count is: " + swBSurfParam.ControlPointRowCount);
            Debug.Print(" Control point dimension is: " + swBSurfParam.ControlPointDimension);
            Debug.Print(" U periodicity is: " + swBSurfParam.UPeriodicity);
            Debug.Print(" V periodicity is: " + swBSurfParam.VPeriodicity);

            UKnots = swBSurfParam.UKnots;
            double[] uknots;
            uknots = (double[])UKnots;
            Debug.Print("Knot vector in the U direction: ");
        }
    }
}
```

```

    for (i = 0; i <= uknots.GetUpperBound(0); i++)
    {
        Debug.Print(uknots[i].ToString());
    }
    VKnots = swBSurfParam.VKnots;
    double[] vknots;
    vknots = (double[])VKnots;
    Debug.Print("Knot vector in the V direction: ");
    for (i = 0; i <= vknots.GetUpperBound(0); i++)
    {
        Debug.Print(vknots[i].ToString());
    }

    // Get control points in row = 2, column = 3 of the control point matrix
    vCtrlPts = swBSurfParam.GetControlPoints(2, 3);
    double[] ctrlpts;
    ctrlpts = (double[])vCtrlPts;
    Debug.Print("Control point at row=2 and column=3: ");
    for (i = 0; i <= ctrlpts.GetUpperBound(0); i++)
    {
        Debug.Print(ctrlpts[i].ToString());
    }
}

public SldWorks swApp;
}
}

```

Get B-Spline Surface Parameterization Data Example (VBA)

This example shows how to get B-spline parameterization data for a selected face or surface.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
' 2. Select a face.
' 3. Open the Immediate Window.
'
' Postconditions: Examine the Immediate window.
'-----

Option Explicit

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr              As SldWorks.SelectionMgr
    Dim swFace               As SldWorks.Face2
    Dim swSurf               As SldWorks.Surface
    Dim swSurfParam           As SldWorks.SurfaceParameterizationData
    Dim swBSurfParam          As SldWorks.BSurfParamData
    Dim sense                As Boolean
    Dim UKnots               As Variant
    Dim VKnots               As Variant
    Dim vCtrlPts              As Variant
    Dim i                    As Integer

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject5(1)
    Set swSurf = swFace.GetSurface
    Set swSurfParam = swSurf.Parameterization2
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Surface:"
    Debug.Print " U minimum is: " & swSurfParam.UMin
    Debug.Print " U minimum bound type is: " & swSurfParam.UMinBoundType
    Debug.Print " U maximum is: " & swSurfParam.UMax
    Debug.Print " U maximum bound type is: " & swSurfParam.UMaxBoundType
    Debug.Print " U property count is: " & swSurfParam.UPropertyNumber
    Dim varUProperties As Variant
    varUProperties = swSurfParam.UProperties
    Debug.Print " U properties: "
    For i = 0 To UBound(varUProperties)
        Debug.Print " " & varUProperties(i)
    Next i

    Debug.Print " V minimum is: " & swSurfParam.VMin
    Debug.Print " V minimum bound type is: " & swSurfParam.VMinBoundType
    Debug.Print " V maximum is: " & swSurfParam.VMax
    Debug.Print " V maximum bound type is: " & swSurfParam.VMaxBoundType
    Debug.Print " V property count is: " & swSurfParam.VPropertyNumber
    Dim varVProperties As Variant
    varVProperties = swSurfParam.VProperties
    Debug.Print " V properties: "
    For i = 0 To UBound(varVProperties)
        Debug.Print " " & varVProperties(i)
    Next i

    Set swBSurfParam = swSurf.GetBSurfParams3(False, False, swSurfParam, 0.01, sense)

    Debug.Print "U order is: " & swBSurfParam.UOrder
    Debug.Print "V order is: " & swBSurfParam.VOrder
    Debug.Print " Control point column count is: " & swBSurfParam.ControlPointColumnCount
    Debug.Print " Control point row count is: " & swBSurfParam.ControlPointRowCount
    Debug.Print " Control point dimension is: " & swBSurfParam.ControlPointDimension
    Debug.Print " U periodicity is: " & swBSurfParam.UPeriodicity
    Debug.Print " V periodicity is: " & swBSurfParam.VPeriodicity

    UKnots = swBSurfParam.UKnots
    Debug.Print "Knot vector in the U direction: "
    For i = 0 To UBound(UKnots)
        Debug.Print UKnots(i)
    Next i
    VKnots = swBSurfParam.VKnots
    Debug.Print "Knot vector in the V direction: "
    For i = 0 To UBound(VKnots)
        Debug.Print VKnots(i)
    Next i

    ' Get control points in row = 2, column = 3 of the control point matrix
    vCtrlPts = swBSurfParam.GetControlPoints(2, 3)
    Debug.Print "Control point at row=2 and column=3: "
```

```
    For i = 0 To UBound(vCtrlPts)
        Debug.Print vCtrlPts(i)
    Next i

End Sub
```

Get B-Spline Surface Parameterization Data Example (VB.NET)

This example shows how to get B-spline parameterization data for a selected face or surface.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
' 2. Select a face.
' 3. Open the Immediate Window.
'-----
' Postconditions: Examine the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
Sub main()
```

```
Dim swModel As ModelDoc2
```

```
Dim swSelMgr As SelectionMgr
```

```
Dim swFace As Face2
```

```
Dim swSurf As Surface
```

```
Dim swSurfParam As SurfaceParameterizationData
```

```
Dim swBSurfParam As BSurfParamData
```

```
Dim sense As Boolean
```

```
Dim UKnots As Object
```

```
Dim VKnots As Object
```

```
Dim vCtrlPts As Object
```

```
Dim i As Integer
```

```
swModel = swApp.ActiveDoc
```

```
swSelMgr = swModel.SelectionManager
```

```
swFace = swSelMgr.GetSelectedObject5(1)
```

```
swSurf = swFace.GetSurface
```

```
swSurfParam = swSurf.Parameterization2
```

```
Debug.Print("File = " & swModel.GetPathName)
```

```
Debug.Print(" Surface:")
```

```
Debug.Print(" U minimum is: " & swSurfParam.UMin)
```

```
Debug.Print(" U minimum bound
type is: " & swSurfParam.UMinBoundType)
```

```
Debug.Print(" U maximum is: " & swSurfParam.UMax)
```

```
Debug.Print(" U maximum bound
type is: " & swSurfParam.UMaxBoundType)
```

```
Debug.Print(" U property count
is: " & swSurfParam.UPropertyNumber)
```

```
Dim varUProperties As Object
```

```
varUProperties = swSurfParam.UProperties
```

```
Debug.Print(" U properties: ")
```

```
For i = 0 To UBound(varUProperties)
```

```
    Debug.Print("    " & varUProperties(i))
```

```
Next i
```

```
Debug.Print(" V minimum is: " & swSurfParam.VMin)
```

```
Debug.Print(" V minimum bound
type is: " & swSurfParam.VMinBoundType)
```

```

        Debug.Print(" V maximum is: " & swSurfParam.VMax)

        Debug.Print(" V maximum bound
type is: " & swSurfParam.VMaxBoundType)

        Debug.Print(" V property count
is: " & swSurfParam.VPropertyNumber)

        Dim varVProperties As Object

        varVProperties = swSurfParam.VProperties

        Debug.Print(" V properties: ")

        For i = 0 To UBound(varVProperties)
            Debug.Print("      " & varVProperties(i))
        Next i

        swBSurfParam = swSurf.GetBSurfParams3(False, False, swSurfParam, 0.01, sense)

        Debug.Print("U order is: " & swBSurfParam.UOrder)

        Debug.Print("V order is: " & swBSurfParam.VOrder)

        Debug.Print(" Control point
column count is: " & swBSurfParam.ControlPointColumnCount)

        Debug.Print(" Control point row
count is: " & swBSurfParam.ControlPointRowCount)

        Debug.Print(" Control point
dimension is: " & swBSurfParam.ControlPointDimension)

        Debug.Print(" U periodicity is: " & swBSurfParam.UPeriodicity)

        Debug.Print(" V periodicity is: " & swBSurfParam.VPeriodicity)

        UKnots = swBSurfParam.UKnots

        Debug.Print("Knot vector in the U
direction: ")

        For i = 0 To UBound(UKnots)
            Debug.Print(UKnots(i))
        Next i

        VKnots = swBSurfParam.VKnots

        Debug.Print("Knot vector in the V
direction: ")

        For i = 0 To UBound(VKnots)
            Debug.Print(VKnots(i))
        Next i

        ' Get control points in row = 2,
        column = 3 of the control point matrix

        vCtrlPts =
swBSurfParam.GetControlPoints(2, 3)

        Debug.Print("Control point at
row=2 and column=3: ")

        For i = 0 To UBound(vCtrlPts)
            Debug.Print(vCtrlPts(i))
        Next i

    End Sub

    Public swApp As SldWorks

End Class

```

Get Base Views Example (VBA)

This example shows how to get the names of the base views for all of the views in a drawing.

```
'-----
'
' Preconditions: Drawing document is open.
'
' Postconditions: None
'
'-----

Option Explicit
Sub main()
    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swDraw           As SldWorks.DrawingDoc
    Dim swView           As SldWorks.View
    Dim swBaseView       As SldWorks.View
    Dim bRet             As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swView = swDraw.GetFirstView

    Debug.Print "File = " & swModel.GetPathName

    Do While Not swView Is Nothing
        ' Returns NULL if no parent
        Set swBaseView = swView.GetBaseView

        Debug.Print " " & swView.Name

        If Not swBaseView Is Nothing Then
            Debug.Print " --> " & swBaseView.Name
        End If

        Set swView = swView.GetNextView
    Loop
End Sub
'-----
```

Get BCurve Spline Points Example (C#)

This example shows how to get B-spline parameter data (knots, control points, etc.) for a selected spline or curve.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
// 2. Select a curve.
// 3. Open the Immediate Window.
//
// Postconditions: Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;

using SolidWorks.Interop.swconst;

using System;

using System.Diagnostics;

namespace GetBSplineParamData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = null;

            Part = (ModelDoc2)swApp.ActiveDoc;

            SelectionMgr swSelectMgr = null;

            swSelectMgr = (SelectionMgr)Part.SelectionManager;

            IEdge swSketchSeg = null;

            swSketchSeg = (ISketchSegment)swSelectMgr.GetSelectedObject6(1,-1);

            Curve swCurveIn = null;

            SplineParamData swSplineParaData = default(SplineParamData);

            object varCtrlPoints = null;

            object varKnotPoints = null;

            bool boolStatus = false;

            int i = 0;

            swCurveIn = (Curve)swSketchSeg.GetCurve();

            Debug.Print("");

            Debug.Print("B-spline");

            Debug.Print("");

            swSplineParaData = (SplineParamData)swCurveIn.GetBCurveParams5(false, false, true, true);

            Debug.Print("The dimension is: " + swSplineParaData.Dimension);

            Debug.Print("The order is: " + swSplineParaData.Order);

            Debug.Print("The periodicity is: " + swSplineParaData.Periodic);

            Debug.Print("The control point count is: " + swSplineParaData.ControlPointsCount);

            Debug.Print("The knot point count is: " + swSplineParaData.KnotPointsCount);

            Debug.Print("COLORREF of spline: " + swSplineParaData.Color);

            Debug.Print("Layer of spline: " + swSplineParaData.Layer);

            Debug.Print("Style overrides as defined in swLayerOverride_e: " + swSplineParaData.LayerOverride);

            Debug.Print("Line style as defined in swLineStyles_e: " + swSplineParaData.LineStyle);
        }
    }
}
```

```
        Debug.Print("Line weight at defined in swLineWeights_e: " + swSplineParaData.LineWidth);  
    }  
  
    public SldWorks swApp;  
}  
}
```

Get BCurve Spline Points Example (VBA)

This example shows how to get B-spline parameter data (knots, control points, etc.) for a selected spline or curve.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
' 2. Select a curve.
' 3. Open the Immediate Window.
'
' Postconditions: Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks

    Dim Part As SldWorks.ModelDoc2

    Set swApp = CreateObject("SldWorks.Application")
    Set Part = swApp.ActiveDoc

    Dim swSelectMgr As SldWorks.SelectionMgr
    Set swSelectMgr = Part.SelectionManager

    Dim swSketchSeg As SldWorks.Edge
    Set swSketchSeg = swSelectMgr.GetSelectedObject6(1, -1)

    Dim swCurveIn As SldWorks.Curve
    Dim swSplineParaData As SldWorks.SplineParamData

    Dim boolStatus As Boolean
    Dim i As Long

    Set swCurveIn = swSketchSeg.GetCurve

    Debug.Print ""
    Debug.Print "B-spline"
    Debug.Print ""
    Set swSplineParaData = swCurveIn.GetBCurveParams5(False, False, True, True)
    Debug.Print "The dimension is: " & swSplineParaData.Dimension
    Debug.Print "The order is: " & swSplineParaData.Order
    Debug.Print "The periodicity is: " & swSplineParaData.Periodic
    Debug.Print "The control point count is: " & swSplineParaData.ControlPointsCount
    Debug.Print "The knot point count is: " & swSplineParaData.KnotPointsCount

    Debug.Print "COLORREF of spline: " & swSplineParaData.Color
    Debug.Print "Layer of spline: " & swSplineParaData.Layer
    Debug.Print "Style overrides as defined in swLayerOverride_e: " & swSplineParaData.LayerOverride
    Debug.Print "Line style as defined in swLineStyles_e: " & swSplineParaData.LineStyle
    Debug.Print "Line weight at defined in swLineWeights_e: " & swSplineParaData.LineWidth

End Sub
```

Get BCurve Spline Points Example (VB.NET)

This example shows how to get B-spline parameter data (knots, control points, etc.) for a selected spline or curve.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\molds\telephone.sldprt.
' 2. Select a curve.
' 3. Open the Immediate Window.
'
' Postconditions: Examine the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim Part As ModelDoc2
        Part = swApp.ActiveDoc

        Dim swSelectMgr As SelectionMgr
        swSelectMgr = Part.SelectionManager

        Dim swSketchSeg As Edge
        swSketchSeg =
        swSelectMgr.GetSelectedObject6(1, -1)

        Dim swCurveIn As Curve
        Dim swSplineParaData As SplineParamData
        Dim varCtrlPoints As Object = Nothing
        Dim varKnotPoints As Object = Nothing
        Dim boolStatus As Boolean
        Dim i As Long

        swCurveIn = swSketchSeg.GetCurve

        Debug.Print("")
        Debug.Print("B-spline")
        Debug.Print("")
        swSplineParaData = swCurveIn.GetBCurveParams5(False, False, True, True)
        Debug.Print("The dimension is: " & swSplineParaData.Dimension)
        Debug.Print("The order is: " & swSplineParaData.Order)
        Debug.Print("The periodicity is:
" & swSplineParaData.Periodic)
        Debug.Print("The control point
count is: " & swSplineParaData.ControlPointsCount)

        Debug.Print("The knot point count
is: " & swSplineParaData.KnotPointsCount)

        Debug.Print("COLORREF of spline:
" & swSplineParaData.Color)

        Debug.Print("Layer of spline: " & swSplineParaData.Layer)

        Debug.Print("Style overrides as
defined in swLayerOverride_e: " & swSplineParaData.LayerOverride)

        Debug.Print("Line style as
defined in swLineStyles_e:: " & swSplineParaData.LineStyle)
    
```

```
        Debug.Print("Line weight at  
defined in swLineWeights_e: " & swSplineParaData.LineWidth)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Get Bend Line Note Values Example (VBA)

This example shows how to get bend line note values.

```
'-----
' Preconditions:
' 1. Open a drawing document containing views of a sheet metal
'    flat pattern configuration with bend line notes.
' 2. Select a bend line note.
'
' Postconditions: Prints the bend line note value to the
' the Immediate window.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swNote As SldWorks.Note
Dim startMathPoint As SldWorks.MathPoint
Dim endMathPoint As SldWorks.MathPoint
Dim UpOrDown As Boolean
Dim Angle As Double
Dim Radius As Double
Dim IsBendNote As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    Set swNote = swSelMgr.GetSelectedObject6(1, -1)

    IsBendNote = swNote.GetBendLineValues(UpOrDown, Angle, Radius, startMathPoint, endMathPoint)

    Debug.Print "Up (True) or Down (False): " & UpOrDown
    ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Debug.Print "Angle: " & Angle * 57.3 & " degrees"
    Debug.Print "Radius: " & Radius * 1000# & " mm"
End Sub
```

Get Block Definitions in Part or Assembly Example (VBA)

This example shows how to get all of the block definitions and their block instances in a part or assembly.

```
'-----  
'  
' Preconditions: Part document is open and contains at least  
'           one block definition and block instance.  
'  
' Postconditions: All of the block instances in the part are selected.  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim skMgr As SldWorks.SketchManager  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim selBlockInst As SldWorks.SketchBlockInstance  
Dim pBlock As SldWorks.SketchBlockDefinition  
Dim plnst As Object  
Dim vBlocks As Variant  
Dim vInstances As Variant  
Dim itr As Long  
Dim jtr As Long  
Dim retVal As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    Set skMgr = swModel.SketchManager  
  
    vBlocks = skMgr.GetSketchBlockDefinitions  
  
    ' Exit if no block definitions  
    If IsEmpty(vBlocks) Then  
        Exit Sub  
    End If  
  
    ' Process blocks instances  
    For itr = 0 To UBound(vBlocks)  
        Set pBlock = vBlocks(itr)  
        vInstances = pBlock.GetInstances  
        If False = IsEmpty(vInstances) Then  
            For jtr = 0 To UBound(vInstances)  
                Set plnst = vInstances(jtr)  
                retVal = plnst.Select(True, Nothing)  
                Set selBlockInst = swSelMgr.GetSelectedObject6(1, -1)  
            Next jtr  
        End If  
    Next itr  
End Sub
```

Get Block Information Example (VBA)

This example shows how to get information about blocks in a drawing document. This example also shows how to migrate the now obsolete and not-supported block definition and block instance interfaces (IBlockDefinition and IBlockInstance) to the new block definition and sketch block instance interfaces (ISketchBlockDefinition and ISketchBlockInstance).

```
'-----
' Preconditions:
' 1. Open drawing with block definitions
'    and instances.
' 2. Open the Immediate window.
' Postconditions: Examine the Immediate window.
'-----

Option Explicit
Sub main()

    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swDraw           As SldWorks.DrawingDoc
    Dim vBlockDef         As Variant
    Dim vBlockInst        As Variant

    '-----

    ' OLD BLOCKS: Obsolete and not supported block
    '    interfaces as of SOLIDWORKS 2007
    'Dim swBlockDef       As SldWorks.BlockDefinition
    'Dim swBlockInst      As SldWorks.BlockInstance
    '-----

    '-----

    ' NEW BLOCKS: New block interfaces as of SOLIDWORKS 2007
    Dim swBlockDef       As SldWorks.SketchBlockDefinition
    Dim swBlockInst      As SldWorks.SketchBlockInstance
    '-----

    Dim vNote            As Variant
    Dim swNote           As SldWorks.Note
    Dim vDispDim         As Variant
    Dim swDispDim        As SldWorks.DisplayDimension
    Dim swDim            As SldWorks.Dimension
    Dim i                As Long
    Dim j                As Long
    Dim k                As Long
    Dim bRet             As Boolean

    '-----

    ' NEW BLOCKS: Additional declaration needed for new blocks
    Dim SwSketchMgr       As SldWorks.SketchManager
    '-----

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    '-----

    ' NEW BLOCKS: For SketchBlockDefinition
    Set SwSketchMgr = swModel.SketchManager
    '-----

    Debug.Print "File = " & swModel.GetPathName

    '-----

    ' OLD BLOCKS: Obsolete and not supported method to get block
    '    definitions
    'vBlockDef = swDraw.GetBlockDefinitions
    '-----

    '-----

    ' NEW BLOCKS: New method to get block definitions
    vBlockDef = SwSketchMgr.GetSketchBlockDefinitions
    '-----
```

```

If Not IsEmpty(vBlockDef) Then
    For i = 0 To UBound(vBlockDef)
        Set swBlockDef = vBlockDef(i)

        vBlockInst = swBlockDef.GetInstances
        vNote = swBlockDef.GetNotes
        vDispDim = swBlockDef.GetDisplayDimensions

        =====

        ' OLD BLOCKS: Obsolete and not supported block-related
        '     methods and properties
        'Debug.Print " " & swBlockDef.Name
        'Debug.Print "   UseExternalFile = " & swBlockDef.GetUseExternalFile
        'Debug.Print "   ExternalFileName = " & swBlockDef.GetExternalFileName
        =====

        =====

        ' NEW BLOCKS: New block-related properties for external files; no
        ' corresponding block definition Name property
        Debug.Print "   Block definition linked to file? " & swBlockDef.LinkToFile
        Debug.Print "   File name: " & swBlockDef.FileName
        =====

    If Not IsEmpty(vNote) Then
        Debug.Print ""
        Debug.Print "   Notes:"
        For j = 0 To UBound(vNote)
            Set swNote = vNote(j)

            Debug.Print "   Tag Name       = " & swNote.TagName
            Debug.Print "   Text           = " & swNote.GetText
            Debug.Print ""

        Next j
    End If

    If Not IsEmpty(vDispDim) Then
        Debug.Print ""
        Debug.Print "   Dimensions:"
        For j = 0 To UBound(vDispDim)
            Set swDispDim = vDispDim(j)
            Set swDim = swDispDim.GetDimension

            Debug.Print "   Name           = " & swDim.Name
            Debug.Print "   FullName        = " & swDim.FullName
            Debug.Print "   Type            = " & swDim.GetType
            Debug.Print "   DrivenState      = " & swDim.DrivenState
            Debug.Print "   ReadOnly         = " & swDim.ReadOnly
            Debug.Print "   Value            = " & swDim.GetSystemValue2("") * 1000# & " mm"
            Debug.Print "   TextAll          = " & swDispDim.GetText(swDimensionTextAll)
            Debug.Print "   TextPrefix       = " & swDispDim.GetText(swDimensionTextPrefix)
            Debug.Print "   TextSuffix       = " & swDispDim.GetText(swDimensionTextSuffix)
            Debug.Print "   CalloutAbove     = " & swDispDim.GetText(swDimensionTextCalloutAbove)
            Debug.Print "   CalloutBelow     = " & swDispDim.GetText(swDimensionTextCalloutBelow)
            Debug.Print ""

        Next j
    End If

    If Not IsEmpty(vBlockInst) Then
        Debug.Print ""
        For j = 0 To UBound(vBlockInst)
            Set swBlockInst = vBlockInst(j)
            vNote = swBlockInst.GetAttributes

            =====

            ' NEW BLOCKS: Get block instance Name
            Debug.Print "   Name of block instance: " & swBlockInst.Name
            =====

            Debug.Print "   Block instance(" & j & "):"

            ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print "   Angle           = " & swBlockInst.Angle * 57.3 & " deg"
        Next j
    End If
End If

```

```

        Debug.Print "    Scale          = " & swBlockInst.Scale
        Debug.Print "    TextDisplay    = " & swBlockInst.TextDisplay
    If Not IsEmpty(vNote) Then
        Debug.Print "        Notes:"
        For k = 0 To UBound(vNote)
            Set swNote = vNote(k)
            Debug.Print "            Tag Name      = " & swNote.TagName
            Debug.Print "            Text          = " & swNote.GetText
            Debug.Print ""
        Next k
    End If
Next j
End If
Debug.Print " -----"

Next i
End If
End Sub
'-----

```

Get Block Instance in Part or Assembly Example (VBA)

This example shows how to get a block instance and its block definition.

```
'-----  
'  
' Precondition:  
'      (1) Part or assembly document is open  
'          that has at least one block instance.  
'      (2) Block instance is selected.  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swBlockInst As SldWorks.SketchBlockInstance  
Dim swBlockDef As SldWorks.SketchBlockDefinition  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swBlockInst = swSelMgr.GetSelectedObject6(1, 0)  
  
Debug.Print "BLOCK INSTANCE: "  
Debug.Print "Number of attributes: " & swBlockInst.GetAttributeCount  
Debug.Print "Scale: " & swBlockInst.Scale  
Debug.Print "Name: " & swBlockInst.Name  
  
Set swBlockDef = swBlockInst.Definition  
Debug.Print ""  
Debug.Print "BLOCK DEFINITION: "  
Debug.Print "Number of arcs: " & swBlockDef.GetArcCount  
Debug.Print "Number of dimensions: " & swBlockDef.GetDisplayDimensionCount  
Debug.Print "Number of lines: " & swBlockDef.GetLineCount  
  
End Sub
```

Get Blocking State Example (VBA)

This example shows how to get the blocking state for a model document.

```
'-----  
'  
' Preconditions: Model document is open.  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
Public Enum swBlockingStates_e  
    swNoBlock = &H0  
    swFullBlock = &H1  
    swModifyBlock = &H2  
    swPartialModifyBlock = &H3  
    swEditorBlock = &H4  
End Enum  
  
Sub main()  
    Dim swApp           As SldWorks.SldWorks  
    Dim swModel          As SldWorks.ModelDoc2  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Debug.Print "BlockingState = " & swModel.GetBlockingState  
End Sub  
'-----
```

Get Blocks Data in Any Document Example (VBA)

This example shows how to get blocks data in part, assembly, and drawing documents.

```
'-----  
'  
' Preconditions: Part, assembly, or drawing document containing  
'     one or more block definitions  
'     and block instances is open.  
'  
' Postconditions: None  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim skMgr As SldWorks.SketchManager  
Dim pBlock As SldWorks.SketchBlockDefinition  
Dim pInst As SldWorks.SketchBlockInstance  
Dim leaderPt As SldWorks.MathPoint  
Dim insPt As SldWorks.MathPoint  
Dim vInstPt As Variant  
Dim vInstances As Variant  
Dim vBlocks As Variant  
Dim nInstanceCount As Long  
Dim itr As Long  
Dim jtr As Long  
Dim bLinkToFile As Boolean  
Dim strInsPoint As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set skMgr = swModel.SketchManager  
  
    vBlocks = skMgr.GetSketchBlockDefinitions  
  
    ' Exit if no blocks  
    If IsEmpty(vBlocks) Then  
        Exit Sub  
    End If  
  
    ' Process block definitions  
    For itr = 0 To UBound(vBlocks)  
  
        Set pBlock = vBlocks(itr)  
  
        ' Block definition linked to file?  
        Debug.Print "  Link To File = " & pBlock.LinkToFile  
  
        ' Block linked file name  
        Debug.Print "  Link File Name = " & pBlock.FileName  
  
        ' Block definition insertion point  
        Set insPt = pBlock.InsertionPoint  
        vInstPt = insPt.ArrayData  
        strInsPoint = "  Insertion point: x = " + CStr(vInstPt(0) * 1000) + " , y = " + CStr(vInstPt(1) * 1000) + " , z = " + CStr(vInstPt(2) * 1000)  
        Debug.Print strInsPoint  
  
        ' Number of block instances of this block definition  
        nInstanceCount = pBlock.GetInstanceCount  
        Debug.Print "  Instance Count = " & nInstanceCount  
  
        ' Process block instances  
        If nInstanceCount > 0 Then
```

```

vInstances = pBlock.GetInstances
For jtr = 0 To UBound(vInstances)
    Set pInst = vInstances(jtr)

    ' Block instance position
    Set insPt = pInst.InstancePosition
    vInstPt = insPt.ArrayData
    strInsPoint = "      Instance position: x = " + CStr(vInstPt(0) * 1000) + " , y = " + CStr(vInstPt(1) * 1000) + " , z = " + CStr(vInstPt(2) * 1000)
    Debug.Print strInsPoint

    ' Block instance angle
    Debug.Print "      Angle = " & pInst.Angle

    ' Block instance scale
    Debug.Print "      Scale = " & pInst.Scale

    ' Block instance owner's sketch name
    Debug.Print "      Owner Sketch = " & pInst.GetSketch.Name

    Next jtr
End If

Next itr

End Sub

```

Get Blocks in Drawing Example (VBA)

This example shows how to get a block definition and its block instances in a drawing.

```
'-----  
'  
' Precondition:  
' (1) Drawing is open that contains at least  
' one block definition with at least one  
' block instance.  
' (2) The block definition is selected in the  
' FeatureManager design tree.  
'  
' Postconditions: None  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swBlockDef As SldWorks.SketchBlockDefinition  
Dim swBlockInst As SldWorks.SketchBlockInstance  
Dim nbrBlockInst As Integer  
Dim vBlockInst As Variant  
Dim itr As Integer  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swFeat = swSelMgr.GetSelectedObject6(1, 0)  
Set swBlockDef = swFeat.GetSpecificFeature2  
  
Debug.Print "Feature type: " & swFeat.GetTypeName  
nbrBlockInst = swBlockDef.GetInstanceCount  
Debug.Print "Number of instances of selected block: " & nbrBlockInst  
  
If nbrBlockInst > 0 Then  
vBlockInst = swBlockDef.GetInstances  
For itr = 0 To (nbrBlockInst - 1)  
Set swBlockInst = vBlockInst(itr)  
Debug.Print "Name of block instances: " & swBlockInst.Name  
Next itr  
End If  
  
End Sub
```

Get Bodies Example (C++ COM)

This example shows how to get the bodies in a part document.

```
// -----
void APITestFunction()
{
    HRESULT                hr = S_OK;
    VARIANT_BOOL           bRet = VARIANT_FALSE;
    long                   nRetVal = -1;
    CComPtr<ISldWorks>      pSldWorks;
    CComPtr<IModelDoc2>     pModel;
    CComQIPtr<IPartDoc>    pPart;
    VARIANT                vBodyArr;
    long                   i = -1;
    // Connect to SOLIDWORKS Design
    pSldWorks = TheApplication->GetSWApp();
    ASSERT(pSldWorks);
    hr = pSldWorks->get_IActiveDoc2(&pModel);
    ASSERT(pModel);
    hr = pModel->ClearSelection2(VARIANT_TRUE);
    pPart = pModel;
    ASSERT(pPart);
    hr = pPart->GetBodies2(swSolidBody, VARIANT_TRUE, &vBodyArr);
    ASSERT(NULL != vBodyArr.parray);
    // Process bodies
    SAFEARRAY*             psaBody = V_ARRAY(&vBodyArr);
    LPDISPATCH*           pBodyDispArray = NULL;
    long                   nBodyHighIndex = -1;
    long                   nBodyCount = -1;
    hr = SafeArrayAccessData(psaBody, (void **) &pBodyDispArray);
    ASSERT(S_OK == hr);
    ASSERT(NULL != pBodyDispArray);
    // Get index number of highest array element
    // The array range is from 0 to highIndex
    hr = SafeArrayGetUBound(psaBody, 1, &nBodyHighIndex);
    ASSERT(S_OK == hr);
    // Actual number of array elements is nBodyHighIndex + 1
    nBodyCount = nBodyHighIndex + 1;
    // Process each body
    for (i = 0; i < nBodyCount; i++)
    {
        CComQIPtr<IBody2>  pBody;
        // Calls AddRef() on IDispatch ---> refcount = 2
        pBody = pBodyDispArray[i];
        ASSERT(pBody);
        hr = pBody->Select(VARIANT_TRUE, 0, &bRet);
        ASSERT(bRet);
        // After this call ---> refcount = 1
        // When it goes out of scope ---> refcount = 0
        pBody.Release();
    }
    // Unaccess & destroy the component SafeArray
    hr = SafeArrayUnaccessData(psaBody);
    hr = SafeArrayDestroy(psaBody);
    return;
} //void APITestFunction()
// -----
```

Get Bodies in Body Folders Example (C#)

This example shows how to get the bodies in body folder features.

```
//-----
// Preconditions: Open a part document that
// contains a body folder that contains
// one or more solid, surface, cut-list,
// or weldment items.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Windows.Forms;

namespace BodyFolderType_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swPart;
        Feature swFeat;

        string[] BodyFolderType = new string[6];

        public void Main()
        {
            BodyFolderType[0] = "dummy";
            BodyFolderType[1] = "swSolidBodyFolder";
            BodyFolderType[2] = "swSurfaceBodyFolder";
            BodyFolderType[3] = "swBodySubFolder";
            BodyFolderType[4] = "swWeldmentSubFolder";
            BodyFolderType[5] = "swWeldmentCutListFolder";

            swPart = (ModelDoc2)swApp.ActiveDoc;

            Debug.Print("File = " + swPart.GetPathName());

            swFeat = (Feature)swPart.FirstFeature();
            TraverseFeatures(swFeat, true);

        }

        public void DoTheWork(Feature thisFeat)
        {
            bool IsBodyFolder = false;
            IsBodyFolder = false;

            string FeatType = null;
            FeatType = thisFeat.GetTypeName();

            if (FeatType == "SolidBodyFolder" | FeatType == "SurfaceBodyFolder" | FeatType == "CutListFolder" | FeatType ==
"SubWeldFolder" | FeatType == "SubAtomFolder")
            {
                IsBodyFolder = true;
            }

            if (IsBodyFolder)
            {
                BodyFolder BodyFolder = default(BodyFolder);
                BodyFolder = (BodyFolder)thisFeat.GetSpecificFeature2();

                int BodyFolderTypeE = 0;
                BodyFolderTypeE = BodyFolder.Type;
                Debug.Print(thisFeat.Name + "," + FeatType + "," + BodyFolderType[BodyFolderTypeE] + " type " + BodyFolderTypeE);

                int BodyCount = 0;
                BodyCount = BodyFolder.GetBodyCount();

                Debug.Print("    Body Count is " + BodyCount);

                object[] vBodies = null;
                vBodies = (object[])BodyFolder.GetBodies();

                int i = 0;

                if ((vBodies != null))
                {
                    for (i = vBodies.GetLowerBound(0); i <= vBodies.GetUpperBound(0); i++)
                    {
```

```

        Body2 Body = default(Body2);
        Body = (Body2)vBodies[i];
        Debug.Print("    " + Body.Name);
    }
}

Feature FeatureFromBodyFolder = default(Feature);
FeatureFromBodyFolder = BodyFolder.GetFeature();

if ((!object.ReferenceEquals(FeatureFromBodyFolder, thisFeat)))
{
    MessageBox.Show("Features don't match!");
}

}
else
{
}
}

}

public void TraverseFeatures(Feature thisFeat, bool isTopLevel)
{
    Feature curFeat = default(Feature);
    curFeat = thisFeat;

    while ((curFeat != null))
    {
        DoTheWork(curFeat); //Do the thing that we are doing this feature traversal for

        Feature subfeat = default(Feature);
        subfeat = (Feature)curFeat.GetFirstSubFeature();

        while ((subfeat != null))
        {
            TraverseFeatures(subfeat, false);
            Feature nextSubFeat = default(Feature);
            nextSubFeat = (Feature)subfeat.GetNextSubFeature();
            subfeat = nextSubFeat;
            nextSubFeat = null;
        }

        subfeat = null;

        Feature nextFeat = default(Feature);

        if (isTopLevel)
        {
            nextFeat = (Feature)curFeat.GetNextFeature();
        }
        else
        {
            nextFeat = null;
        }

        curFeat = nextFeat;
        nextFeat = null;
    }
}

}

public SldWorks swApp;

}

}

```

Get Bodies in Body Folders Example (VBA)

This example shows how to get the bodies in body folder features.

```

'-----
' Preconditions: Open a part document that
' contains a body folder that contains
' one or more solid, surface, cut-list,
' or weldment items.
'
' Postconditions: Inspect the Immediate window.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim Indent As Long
Dim BodyFolderType(5) As String

Sub main()
    BodyFolderType(0) = "dummy"
    BodyFolderType(1) = "swSolidBodyFolder"
    BodyFolderType(2) = "swSurfaceBodyFolder"
    BodyFolderType(3) = "swBodySubFolder"
    BodyFolderType(4) = "swWeldmentSubFolder"
    BodyFolderType(5) = "swWeldmentCutListFolder"

    Set swApp = Application.SldWorks
    Set swPart = swApp.ActiveDoc

    Debug.Print "File = " & swPart.GetPathName
    Indent = -3
    Set swFeat = swPart.FirstFeature
    TraverseFeatures swFeat, True
End Sub

Sub DoTheWork(thisFeat As SldWorks.Feature)
    Dim IsBodyFolder As Boolean
    IsBodyFolder = False

    Dim FeatType As String
    FeatType = thisFeat.GetTypeName

    If FeatType = "SolidBodyFolder" Or FeatType = "SurfaceBodyFolder" Or FeatType = "CutListFolder" Or FeatType = "SubWeldFolder"
    Or FeatType = "SubAtomFolder" Then
        IsBodyFolder = True
    End If

    If IsBodyFolder Then
        Debug.Print Format(String(Indent, " ") & thisFeat.Name, "!" & String(40, "@")); Format(FeatType, "!" & String(30, "@"));
        Dim BodyFolder As SldWorks.BodyFolder
        Set BodyFolder = thisFeat.GetSpecificFeature2

        Dim BodyFolderTypeE As Long
        BodyFolderTypeE = BodyFolder.Type

        Debug.Print Format(BodyFolderType(BodyFolderTypeE), "!" & String(30, "@")); Format(BodyFolderTypeE, "!" & String(10, "@"));

        Dim BodyCount As Long
        BodyCount = BodyFolder.GetBodyCount

        Debug.Print "Body Count is " & BodyCount

        Dim vBodies As Variant
        vBodies = BodyFolder.GetBodies

        Dim i As Long
        If Not IsEmpty(vBodies) Then
            For i = LBound(vBodies) To UBound(vBodies)
                Dim Body As SldWorks.Body2
                Set Body = vBodies(i)
                Debug.Print Format(String(Indent + 3, " ") & Body.Name, "!" & String(30, "@"))
            Next i
        End If

        Dim FeatureFromBodyFolder As SldWorks.Feature
        Set FeatureFromBodyFolder = BodyFolder.GetFeature

        If Not FeatureFromBodyFolder Is thisFeat Then
            MsgBox "Features don't match!"
        End If
    Else
        End If
    End Sub

Sub TraverseFeatures(thisFeat As SldWorks.Feature, isTopLevel As Boolean)
    Dim curFeat As SldWorks.Feature
    Set curFeat = thisFeat
    Indent = Indent + 3

```

```

While Not curFeat Is Nothing
  DoTheWork curFeat 'Do the thing that we are doing this feature traversal for
  Dim subfeat As SldWorks.Feature
  Set subfeat = curFeat.GetFirstSubFeature

  While Not subfeat Is Nothing
    TraverseFeatures subfeat, False
    Dim nextSubFeat As SldWorks.Feature
    Set nextSubFeat = subfeat.GetNextSubFeature
    Set subfeat = nextSubFeat
    Set nextSubFeat = Nothing
  Wend

  Set subfeat = Nothing
  Dim nextFeat As SldWorks.Feature
  If isTopLevel Then
    Set nextFeat = curFeat.GetNextFeature
  Else
    Set nextFeat = Nothing
  End If

  Set curFeat = nextFeat
  Set nextFeat = Nothing
Wend
Indent = Indent - 3
End Sub

```

Get Bodies in Body Folders Example (VB.NET)

This example shows how to get the bodies in body folder features.

```

'-----
' Preconditions: Open a part document that
' contains a body folder that contains
' one or more solid, surface, cut-list,
' or weldment items.
' Postconditions: Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swPart As ModelDoc2
    Dim swFeat As Feature
    Dim BodyFolderType(5) As String

```

Sub main()

```

    BodyFolderType(0) = "dummy"
    BodyFolderType(1) = "swSolidBodyFolder"
    BodyFolderType(2) = "swSurfaceBodyFolder"
    BodyFolderType(3) = "swBodySubFolder"
    BodyFolderType(4) = "swWeldmentSubFolder"
    BodyFolderType(5) = "swWeldmentCutListFolder"

```

```

    swPart = swApp.ActiveDoc

```

```

    Debug.Print("File = " & swPart.GetPathName)

```

```

    swFeat = swPart.FirstFeature
    TraverseFeatures(swFeat, True)

```

End Sub

Sub DoTheWork(ByVal thisFeat As Feature)

```

    Dim IsBodyFolder As Boolean
    IsBodyFolder = False

```

```

    Dim FeatType As String
    FeatType = thisFeat.GetTypeName

```

```

    If FeatType = "SolidBodyFolder" Or FeatType = "SurfaceBodyFolder" Or FeatType = "CutListFolder" Or FeatType =
"SubWeldFolder" Or FeatType = "SubAtomFolder" Then
        IsBodyFolder = True
    End If

```

If IsBodyFolder Then

```

    Dim BodyFolder As BodyFolder
    BodyFolder = thisFeat.GetSpecificFeature2

```

```

    Dim BodyFolderTypeE As Long
    BodyFolderTypeE = BodyFolder.Type
    Debug.Print(thisFeat.Name & ", " & FeatType & ", " & BodyFolderType(BodyFolderTypeE) & " type " & BodyFolderTypeE)

```

```

    Dim BodyCount As Integer
    BodyCount = BodyFolder.GetBodyCount

```

```

    Debug.Print("    Body Count is " & BodyCount)

```

```

    Dim vBodies As Object
    vBodies = BodyFolder.GetBodies

```

```

    Dim i As Integer

```

```

    If Not IsNothing(vBodies) Then
        For i = LBound(vBodies) To UBound(vBodies)
            Dim Body As Body2
            Body = vBodies(i)
            Debug.Print("    " & Body.Name)
        Next i
    End If

```

```

    Dim FeatureFromBodyFolder As Feature
    FeatureFromBodyFolder = BodyFolder.GetFeature

```

```

    If Not FeatureFromBodyFolder Is thisFeat Then
        MsgBox("Features don't match!")
    End If

```

```

        End If
    Else
        End If
    End Sub

Sub TraverseFeatures(ByVal thisFeat As Feature, ByVal isTopLevel As Boolean)

    Dim curFeat As Feature
    curFeat = thisFeat

    While Not curFeat Is Nothing
        DoTheWork(curFeat) 'Do the thing that we are doing this feature traversal for

        Dim subfeat As Feature
        subfeat = curFeat.GetFirstSubFeature

        While Not subfeat Is Nothing
            TraverseFeatures(subfeat, False)
            Dim nextSubFeat As Feature
            nextSubFeat = subfeat.GetNextSubFeature
            subfeat = nextSubFeat
            nextSubFeat = Nothing
        End While

        subfeat = Nothing

        Dim nextFeat As Feature

        If isTopLevel Then
            nextFeat = curFeat.GetNextFeature
        Else
            nextFeat = Nothing
        End If

        curFeat = nextFeat
        nextFeat = Nothing

    End While

End Sub

Public swApp As SldWorks

End Class

```

Get Bodies in Components Example (C++ COM)

This example shows how to get the bodies in assembly components.

```
// -----
void CSwAddin2::GetBodiesOfComponents()
{
    HRESULT hr = S_OK;
    VARIANT_BOOL bRet = VARIANT_FALSE;
    long nRetVal = -1;
    CComPtr <IModelDoc2> pModel;
    CComQIPtr <IAssemblyDoc> pAssembly;
    CComQIPtr <IComponent2> pComponent;
    CComQIPtr <IBody2> pBody;
    VARIANT vComponents;
    VARIANT vBodyArr;
    VARIANT vBodyInfo;
    long Count;
    long nCopmCount;
    long CntBody;
    long BodyCount;

    SAFEARRAY* psaBody;

    hr = iSwApp->get_IActiveDoc2(&pModel);
    ASSERT(pModel);

    pAssembly = pModel;

    ASSERT(pAssembly);

    hr = pAssembly->GetComponents(VARIANT_TRUE,&vComponents);

    ASSERT(NULL!= vComponents.pparray);

    SAFEARRAY* psaComp = V_ARRAY(&vComponents);

    LPDISPATCH* pCompDispArray = NULL;

    hr = SafeArrayAccessData(psaComp,(void**)&pCompDispArray);

    hr = SafeArrayGetUBound(psaComp,1,&Count);

    nCopmCount = Count+1;

    for(int i = 0; i < nCopmCount; i++)
    {
        CComBSTR sCompMateName (L"");

        CComBSTR sBodyName (L"");

        CComQIPtr <IComponent2> pComp;

        pComp = pCompDispArray[i];

        pComp->get_Name2(&sCompMateName);

        pComp->GetBodies3(0,&vBodyInfo,&vBodyArr);
```

```

    ASSERT(NULL!= vBodyArr.parray);

    psaBody = V_ARRAY(&vBodyArr);

    LPDISPATCH* pBodyDispArray = NULL;

    hr = SafeArrayAccessData(psaBody, (void**)&pBodyDispArray);

    hr = SafeArrayGetUBound(psaBody, 1, &CntBody);

    BodyCount = CntBody+1;

    for(int j = 0; j < BodyCount; j++)
    {
        pBody = pBodyDispArray[j];

        pBody->get_Name(&sBodyName);
    }

    hr = SafeArrayUnaccessData(psaBody);
    hr = SafeArrayDestroy(psaBody);
}

hr = SafeArrayUnaccessData(psaComp);
hr = SafeArrayDestroy(psaComp);
}

```

Get Bodies in Components Example (C#)

This example shows how to get the number of normal and user bodies in the components in an assembly.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document
//    to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets and prints each component's name, number of
//    solid bodies, body names, and body types
//    to the Immediate window.
// 2. Right-click filterholder<1> in the FeatureManager
//    design tree and click the Open Part button.
//    Notice that there are no screw holes in the part.
// 3. Close the part and examine the filterholder<1>
//    component, which is the orange, flat, circular
//    component located on the front of the assembly
//    in the graphics area. There are screw
//    holes in the component.
// 4. Examine the filterholder<1>'s information in the
//    Immediate window. Because the component was
//    modified in the assembly, its body is identified
//    as a user body.
//
// NOTE: Because this assembly document is used by
// elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Collections;
namespace GetBodies3Component2CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            object[] vComponents = null;
            Component2 oneComponent = default(Component2);
            Body2 swBody = default(Body2);
            object[] vBodies = null;
            object vBodyInfo;
            int[] BodiesInfo = null;
            int BodyType = 0;
            int errors = 0;
            int warnings = 0;
            int i = 0;
            int j = 0;

            // Open this assembly
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial1\\toolbox\\lens_mount.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swAssembly = (AssemblyDoc)swModel;

            // Get the components in the assembly
            vComponents = (object[])swAssembly.GetComponents(true);
            for (i = 0; i <= vComponents.Length - 1; i++)
            {
                oneComponent = (Component2)vComponents[i];
                Debug.Print(" ");
                Debug.Print("Component name: " + oneComponent.Name2);

                // Get the solid bodies in the component
                vBodies = (object[])oneComponent.GetBodies3((int)swBodyType_e.swSolidBody, out (object)vBodyInfo);
                BodiesInfo = (int[])vBodyInfo;
                Debug.Print(" Number of solid bodies: " + (vBodies.Length + 1));
                for (j = 0; j <= vBodies.Length - 1; j++)
                {
                    Debug.Print(" Body number: " + (j + 1));
                    swBody = (Body2)vBodies[j];
                    Debug.Print(" Body name: " + swBody.Name);

                    // Print the type of body
                    BodyType = (int)BodiesInfo[j];
                    switch (BodyType)
                    {
                        case 0:
                            Debug.Print(" Body type: user");
                            break;
                        case 1:
                            Debug.Print(" Body type: normal");
                            break;
                    }
                }
            }
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Get Bodies in Components Example (VBA)

This example shows how to get the number of normal and user bodies in the components in an assembly.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document
'    to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets and prints each component's name, number of
'    solid bodies, body names, and body types
'    to the Immediate window.
' 2. Right-click filterholder<1> in the FeatureManager
'    design tree and click the Open Part button.
'    Notice that there are no screw holes in the part.
' 3. Close the part and examine the filterholder<1>
'    component, which is the orange, flat, circular
'    component located on the front of the assembly
'    in the graphics area. There are screw
'    holes in the component.
' 4. Examine the filterholder<1>'s information in the
'    Immediate window. Because the component was
'    modified in the assembly, its body is identified
'    as a user body.
'
' NOTE: Because this assembly document is used by
' elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim vComponents As Variant
Dim oneComponent As Component2
Dim vBodies As Variant
Dim vBodyInfo As Variant
Dim errors As Long, warnings As Long
Dim i As Long, j As Long
```

Sub main()

Set swApp = Application.SldWorks

```
    ' Open the assembly
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\toolbox\lens_mount.sldasm", swDocASSEMBLY,
swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssembly = swModel

    ' Get the components in the assembly
    vComponents = swAssembly.GetComponents(True)
    For i = 0 To UBound(vComponents)
        Set oneComponent = vComponents(i)
        Debug.Print " "
        Debug.Print "Component name: " & oneComponent.Name2
        ' Get the solid bodies in the component
        vBodies = oneComponent.GetBodies3(SwConst.swBodyType_e.swSolidBody, vBodyInfo)
        Debug.Print " Number of solid bodies: " & (UBound(vBodies) + 1)
        For j = 0 To UBound(vBodies)
            Debug.Print " Body number: " & (j + 1)
            Debug.Print " Body name: " & vBodies(j).Name
            ' Print the type of body
            Select Case vBodyInfo(j)
                Case 0
                    Debug.Print " Body type: user"
                Case 1
                    Debug.Print " Body type: normal"
            End Select
        Next
    Next
End Sub
```

Get Bodies in Components Example (VB.NET)

This example shows how to get the number of normal and user bodies in the components in an assembly.

```

-----
Preconditions:
1. Verify that the specified assembly document
   to open exists.
2. Open the Immediate window.

Postconditions:
1. Gets and prints each component's name, number of
   solid bodies, body names, and body types
   to the Immediate window.
2. Right-click filterholder<1> in the FeatureManager
   design tree and click the Open Part button.
   Notice that there are no screw holes in the part.
3. Close the part and examine the filterholder<1>
   component, which is the orange, flat, circular
   component located on the front of the assembly
   in the graphics area. There are screw
   holes in the component.
4. Examine the filterholder<1>'s information in the
   Immediate window. Because the component was
   modified in the assembly, its body is identified
   as a user body.

NOTE: Because this assembly document is used by
elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssembly As AssemblyDoc
        Dim vComponents As Object
        Dim oneComponent As Component2
        Dim vBodies As Object
        Dim vBodyInfo As Object = Nothing
        Dim BodyType As Integer
        Dim errors As Integer, warnings As Integer
        Dim i As Integer, j As Integer

        ' Open the assembly
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\toolbox\lens_mount.sldasm", swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swAssembly = swModel

        ' Get the components in the assembly
        vComponents = swAssembly.GetComponents(True)
        For i = 0 To UBound(vComponents)
            oneComponent = vComponents(i)
            Debug.Print(" ")
            Debug.Print("Component name: " & oneComponent.Name2)
            ' Get the solid bodies in the component
            vBodies = oneComponent.GetBodies3(swBodyType_e.swSolidBody, vBodyInfo)
            Debug.Print(" Number of solid bodies: " & (UBound(vBodies) + 1))
            For j = 0 To UBound(vBodies)
                Debug.Print(" Body number: " & (j + 1))
                Debug.Print(" Body name: " & vBodies(j).Name)
                ' Print the type of body
                BodyType = vBodyInfo(j)
                Select Case BodyType
                    Case 0
                        Debug.Print(" Body type: user")
                    Case 1
                        Debug.Print(" Body type: normal")
                End Select
            Next j
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Body Outline Example (C#)

This example shows how to get the outline of a solid body. This example also creates and inserts a sketch of that outline.

```
//-----
// Preconditions: Open a part document that contains
// at least one solid body.
//
// Postconditions: Processes the body outline curves
// to remove gaps before sketching the outline.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetBodyOutline2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        PartDoc swPart;
        ModelDoc2 swModel;
        MathVector swMathVector;
        MathUtility swMathUtility;
        Modeler swModeler;
        object dirVar;
        object[] bVar;
        DispatcherWrapper[] arrBodiesIn = new DispatcherWrapper[1];
        object[] Bodies = new object[1];
        object curveOut;
        object topol;
        object outline;
        double[] sEva;
        double[] eEva;
        double[] sEvaPrev;
        double[] eEvaPrev;
        double[] sEvaNext;
        double[] eEvaNext;
        double[] dirArr = new double[3];
        double s;
        double e;
        int nCt;
        int i;
        int v;
        bool isClosed;
        bool isPer;

        public enum direction
        {
            X = 1,
            Y = 2,
            Z = 3,
            Xminus = 4,
            Yminus = 5,
            Zminus = 6
        }

        public void Main()
        {
            swPart = (PartDoc)swApp.ActiveDoc;
            swModel = (ModelDoc2)swPart;

            //Get the bodies in this part
            bVar = (object[])swPart.GetBodies2((int)swBodyType_e.swSolidBody, false);
            int bdyCnt = bVar.GetLength(0);

            for (i = 0; i < bdyCnt; i++)
            {
                Bodies[i] = bVar[i];
                arrBodiesIn[i] = new DispatcherWrapper(Bodies[i]);
            }

            swModeler = (Modeler)swApp.GetModeler();
            swMathUtility = (MathUtility)swApp.GetMathUtility();

            //Create the direction vector
            dirArr[0] = 0;
            dirArr[1] = 0;
            dirArr[2] = 0;

            direction userDirection = default(direction);
```

```

userDirection = direction.Y;

if (userDirection == direction.X)
{
    dirArr[0] = 1;
}
else if (userDirection == direction.Xminus)
{
    dirArr[0] = -1;
}
else if (userDirection == direction.Y)
{
    dirArr[1] = 1;
}
else if (userDirection == direction.Yminus)
{
    dirArr[1] = -1;
}
else if (userDirection == direction.Z)
{
    dirArr[2] = 1;
}
else if (userDirection == direction.Zminus)
{
    dirArr[2] = -1;
}

dirVar = dirArr;

//Create a MathVector
swMathVector = (MathVector)swMathUtility.CreateVector((dirArr));

//Get the number of curves in the body outline
nCt = swModeler.GetBodyOutline2((arrBodiesIn), swMathVector, 0.00001, true, out curveOut, out topol, out outline);
Object[] crvOut = (Object[])curveOut;

//Open a 3D sketch in the part document
swModel.Insert3DSketch();

//Using the end conditions of the curves, create a 2D sketch of each curve
Curve[] vCurve = null;
int newCt = 0;

for (i = 0; i <= nCt - 1; i++)
{
    ((Curve)crvOut[i]).GetEndParams(out s, out e, out isClosed, out isPer);
    if (((Curve)crvOut[i]).GetLength3(s, e) > 1E-05)
    {
        Array.Resize(ref vCurve, newCt + 1);
        vCurve[newCt] = (Curve)crvOut[i];
        newCt = newCt + 1;
    }
}

double[] sPoints = null;
double[] ePoints = null;

sPoints = new double[(newCt * 3)];
ePoints = new double[(newCt * 3)];

for (i = 0; i <= newCt - 1; i++)
{
    vCurve[i].GetEndParams(out s, out e, out isClosed, out isPer);
    sEva = (double[])vCurve[i].Evaluate(s);
    eEva = (double[])vCurve[i].Evaluate(e);

    if (i > 0)
    {
        v = i - 1;
    }
    else
    {
        v = newCt - 1;
    }

    vCurve[v].GetEndParams(out s, out e, out isClosed, out isPer);
    sEvaPrev = (double[])vCurve[v].Evaluate(s);
    eEvaPrev = (double[])vCurve[v].Evaluate(e);

    if (i < newCt - 1)
    {
        v = i + 1;
    }
    else
    {
        v = 0;
    }

    vCurve[v].GetEndParams(out s, out e, out isClosed, out isPer);
    sEvaNext = (double[])vCurve[v].Evaluate(s);
    eEvaNext = (double[])vCurve[v].Evaluate(e);

    sPoints[i * 3] = sEva[0] + 0.5 * (eEvaPrev[0] - sEva[0]);
    sPoints[i * 3 + 1] = sEva[1] + 0.5 * (eEvaPrev[1] - sEva[1]);
    sPoints[i * 3 + 2] = sEva[2] + 0.5 * (eEvaPrev[2] - sEva[2]);
}

```

```

ePoints[i * 3] = eEva[0] + 0.5 * (sEvaNext[0] - eEva[0]);
ePoints[i * 3 + 1] = eEva[1] + 0.5 * (sEvaNext[1] - eEva[1]);
ePoints[i * 3 + 2] = eEva[2] + 0.5 * (sEvaNext[2] - eEva[2]);

if (userDirection == direction.X | userDirection == direction.Xminus)
{
    sPoints[i * 3] = 0;
    ePoints[i * 3] = 0;
}
else if (userDirection == direction.Y | userDirection == direction.Yminus)
{
    sPoints[i * 3 + 1] = 0;
    ePoints[i * 3 + 1] = 0;
}
else if (userDirection == direction.Z | userDirection == direction.Zminus)
{
    sPoints[i * 3 + 2] = 0;
    ePoints[i * 3 + 2] = 0;
}
}

for (i = 0; i <= (newCt * 3) - 1; i += 3)
{
    swModel.CreateLine2(sPoints[i], sPoints[i + 1], sPoints[i + 2], ePoints[i], ePoints[i + 1], ePoints[i + 2]);
}

//Insert the sketches
swModel.InsertSketch2(true);
swModel.ClearSelection2(true);
}

public SldWorks swApp;
}
}

```

Get Body Outline Example (VBA)

This example shows how to get the outline of a solid body. This example also creates and inserts a sketch of that outline.

```

'-----
' Preconditions: Open a part document that contains
' at least one solid body.
'
' Postconditions: Processes the body outline curves
' to remove gaps before sketching the outline.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc
Dim swModel As SldWorks.ModelDoc2
Dim swMathVector As SldWorks.MathVector
Dim swMathUtility As SldWorks.MathUtility
Dim swModeler As SldWorks.Modeler
Dim dirVar As Variant
Dim bVar As Variant
Dim crvOut As Variant
Dim topol As Variant
Dim outline As Variant
Dim sEva As Variant
Dim eEva As Variant
Dim sEvaPrev As Variant
Dim eEvaPrev As Variant

Dim sEvaNext As Variant
Dim eEvaNext As Variant

Dim dirArr(0 To 2) As Double
Dim s As Double
Dim e As Double
Dim nCt As Long
Dim i As Long
Dim v As Long

Dim isClosed As Boolean
Dim isPer As Boolean

Enum direction
    X = 1
    Y = 2
    Z = 3
    Xminus = 4
    Yminus = 5
    Zminus = 6
End Enum

Sub main()

    Set swApp = Application.SldWorks
    Set swPart = swApp.ActiveDoc
    Set swModel = swPart

    'Get the bodies in this part
    bVar = swPart.GetBodies2(swSolidBody, False)
    Set swModeler = swApp.GetModeler
    Set swMathUtility = swApp.GetMathUtility

    'Create the direction vector
    dirArr(0) = 0
    dirArr(1) = 0
    dirArr(2) = 0

    Dim userDirection As direction
    userDirection = Y

    If userDirection = X Then
        dirArr(0) = 1
    ElseIf userDirection = Xminus Then
        dirArr(0) = -1
    ElseIf userDirection = Y Then
        dirArr(1) = 1
    ElseIf userDirection = Yminus Then
        dirArr(1) = -1
    ElseIf userDirection = Z Then
        dirArr(2) = 1
    ElseIf userDirection = Zminus Then
        dirArr(2) = -1
    End If

    dirVar = dirArr

    'Create a MathVector
    Set swMathVector = swMathUtility.CreateVector((dirArr))

```

```

'Get the number of curves in the body outline
nCt = swModeler.GetBodyOutline2((bVar), swMathVector, 0.00001, True, crvOut, topol, outline)

'Open a 3D sketch in the part document
swPart.Insert3DSketch

'Using the end conditions of the curves, create a 2D sketch of each curve
Dim vCurve() As Curve
Dim newCt As Integer

For i = 0 To nCt - 1
    crvOut(i).GetEndParams s, e, isClosed, isPer
    If crvOut(i).GetLength3(s, e) > 0.00001 Then
        ReDim Preserve vCurve(newCt)
        Set vCurve(newCt) = crvOut(i)

        newCt = newCt + 1
    End If
Next

Dim sPoints() As Double
Dim ePoints() As Double

ReDim sPoints((newCt * 3) - 1)
ReDim ePoints((newCt * 3) - 1)

For i = 0 To newCt - 1
    vCurve(i).GetEndParams s, e, isClosed, isPer
    sEva = vCurve(i).Evaluate(s)
    eEva = vCurve(i).Evaluate(e)

    If i > 0 Then
        v = i - 1
    Else
        v = newCt - 1
    End If

    vCurve(v).GetEndParams s, e, isClosed, isPer
    sEvaPrev = vCurve(v).Evaluate(s)
    eEvaPrev = vCurve(v).Evaluate(e)

    If i < newCt - 1 Then
        v = i + 1
    Else
        v = 0
    End If

    vCurve(v).GetEndParams s, e, isClosed, isPer
    sEvaNext = vCurve(v).Evaluate(s)
    eEvaNext = vCurve(v).Evaluate(e)

    sPoints(i * 3) = sEva(0) + 0.5 * (eEvaPrev(0) - sEva(0))
    sPoints(i * 3 + 1) = sEva(1) + 0.5 * (eEvaPrev(1) - sEva(1))
    sPoints(i * 3 + 2) = sEva(2) + 0.5 * (eEvaPrev(2) - sEva(2))

    ePoints(i * 3) = eEva(0) + 0.5 * (sEvaNext(0) - eEva(0))
    ePoints(i * 3 + 1) = eEva(1) + 0.5 * (sEvaNext(1) - eEva(1))
    ePoints(i * 3 + 2) = eEva(2) + 0.5 * (sEvaNext(2) - eEva(2))

    If userDirection = X Or userDirection = Xminus Then
        sPoints(i * 3) = 0
        ePoints(i * 3) = 0
    ElseIf userDirection = Y Or userDirection = Yminus Then
        sPoints(i * 3 + 1) = 0
        ePoints(i * 3 + 1) = 0
    ElseIf userDirection = Z Or userDirection = Zminus Then
        sPoints(i * 3 + 2) = 0
        ePoints(i * 3 + 2) = 0
    End If
Next i

For i = 0 To (newCt * 3) - 1 Step 3
    swModel.CreateLine2 sPoints(i), sPoints(i + 1), sPoints(i + 2), ePoints(i), ePoints(i + 1), ePoints(i + 2)
Next

'Insert the sketches
swModel.InsertSketch2 True
swModel.ClearSelection2 True

End Sub

```

Get Body Outline Example (VB.NET)

This example shows how to get the outline of a solid body. This example also creates and inserts a sketch of that outline.

```
'-----
' Preconditions: Open a part document that contains
' at least one solid body.
'
```

```
' Postconditions: Processes the body outline curves
' to remove gaps before sketching the outline.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swPart As PartDoc
    Dim swModel As ModelDoc2
    Dim swMathVector As MathVector
    Dim swMathUtility As MathUtility
    Dim swModeler As Modeler
    Dim dirVar As Object
    Dim bVar() As Object
    Dim arrBodiesIn(0) As DispatcherWrapper
    Dim Bodies(0) As Object
    Dim crvOut As Object
    Dim topol As Object
    Dim outline As Object
    Dim sEva As Object
    Dim eEva As Object
    Dim sEvaPrev As Object
    Dim eEvaPrev As Object
    Dim sEvaNext As Object
    Dim eEvaNext As Object
    Dim dirArr(0 To 2) As Double
    Dim s As Double
    Dim e As Double
    Dim nCt As Long
    Dim i As Long
    Dim v As Long
    Dim isClosed As Boolean
    Dim isPer As Boolean
```

```
Enum direction
    X = 1
    Y = 2
    Z = 3
    Xminus = 4
    Yminus = 5
    Zminus = 6
End Enum
```

```
Sub main()
```

```
    swPart = swApp.ActiveDoc
    swModel = swPart
```

```
    'Get the bodies in this part
    bVar = swPart.GetBodies2(swBodyType_e.swSolidBody, False)
```

```
    Dim bdyCnt As Long
    bdyCnt = bVar.Length
```

```
    For i = 0 To i = bdyCnt
        Bodies(i) = bVar(i)
        arrBodiesIn(i) = New DispatcherWrapper(Bodies(i))
    Next
```

```
    swModeler = swApp.GetModeler
    swMathUtility = swApp.GetMathUtility
```

```
    'Create the direction vector
    dirArr(0) = 0
    dirArr(1) = 0
    dirArr(2) = 0
```

```
    Dim userDirection As direction
    userDirection = direction.Y
```

```
    If userDirection = direction.X Then
        dirArr(0) = 1
    ElseIf userDirection = direction.Xminus Then
        dirArr(0) = -1
    ElseIf userDirection = direction.Y Then
```

```

        dirArr(1) = 1
    ElseIf userDirection = direction.Yminus Then
        dirArr(1) = -1
    ElseIf userDirection = direction.Z Then
        dirArr(2) = 1
    ElseIf userDirection = direction.Zminus Then
        dirArr(2) = -1
    End If

    dirVar = dirArr

    'Create a MathVector
    swMathVector = swMathUtility.CreateVector((dirArr))

    'Get the number of curves in the body outline
    nCt = swModeler.GetBodyOutline2((arrBodiesIn), swMathVector, 0.00001, True, crvOut, topol, outline)

    'Open a 3D sketch in the part document
    swPart.Insert3DSketch()

    'Using the end conditions of the curves, create a 2D sketch of each curve
    Dim vCurve() As Curve = Nothing
    Dim newCt As Integer

    For i = 0 To nCt - 1
        crvOut(i).GetEndParams(s, e, isClosed, isPer)
        If crvOut(i).GetLength3(s, e) > 0.00001 Then
            ReDim Preserve vCurve(newCt)
            vCurve(newCt) = crvOut(i)

            newCt = newCt + 1
        End If
    Next

    Dim sPoints() As Double
    Dim ePoints() As Double

    ReDim sPoints((newCt * 3) - 1)
    ReDim ePoints((newCt * 3) - 1)

    For i = 0 To newCt - 1
        vCurve(i).GetEndParams(s, e, isClosed, isPer)
        sEva = vCurve(i).Evaluate(s)
        eEva = vCurve(i).Evaluate(e)

        If i > 0 Then
            v = i - 1
        Else
            v = newCt - 1
        End If

        vCurve(v).GetEndParams(s, e, isClosed, isPer)
        sEvaPrev = vCurve(v).Evaluate(s)
        eEvaPrev = vCurve(v).Evaluate(e)

        If i < newCt - 1 Then
            v = i + 1
        Else
            v = 0
        End If

        vCurve(v).GetEndParams(s, e, isClosed, isPer)
        sEvaNext = vCurve(v).Evaluate(s)
        eEvaNext = vCurve(v).Evaluate(e)

        sPoints(i * 3) = sEva(0) + 0.5 * (eEvaPrev(0) - sEva(0))
        sPoints(i * 3 + 1) = sEva(1) + 0.5 * (eEvaPrev(1) - sEva(1))
        sPoints(i * 3 + 2) = sEva(2) + 0.5 * (eEvaPrev(2) - sEva(2))

        ePoints(i * 3) = eEva(0) + 0.5 * (sEvaNext(0) - eEva(0))
        ePoints(i * 3 + 1) = eEva(1) + 0.5 * (sEvaNext(1) - eEva(1))
        ePoints(i * 3 + 2) = eEva(2) + 0.5 * (sEvaNext(2) - eEva(2))

        If userDirection = direction.X Or userDirection = direction.Xminus Then
            sPoints(i * 3) = 0
            ePoints(i * 3) = 0
        ElseIf userDirection = direction.Y Or userDirection = direction.Yminus Then
            sPoints(i * 3 + 1) = 0
            ePoints(i * 3 + 1) = 0
        ElseIf userDirection = direction.Z Or userDirection = direction.Zminus Then
            sPoints(i * 3 + 2) = 0
            ePoints(i * 3 + 2) = 0
        End If
    Next i

    For i = 0 To (newCt * 3) - 1 Step 3
        swModel.CreateLine2(sPoints(i), sPoints(i + 1), sPoints(i + 2), ePoints(i), ePoints(i + 1), ePoints(i + 2))
    Next

    'Insert the sketches
    swModel.InsertSketch2(True)
    swModel.ClearSelection2(True)

```

End Sub

Public swApp As SldWorks

End Class

Get BOM Balloon Properties Example (C#)

This example shows how to get the properties of a BOM balloon.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open
//    exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Inserts a BOM table.
// 3. Inserts a BOM balloon.
// 4. Gets and sets the BOM balloon's properties.
// 5. Examine the Immediate window.
//
// NOTE: Because this assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace BomTableAnnotationCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Note swNote = default(Note);
            Annotation swAnnotation = default(Annotation);
            object[] attachedEntityArray = null;
            int[] attachedEntityTypeArray = null;
            Entity swEntity = default(Entity);
            Component2 swComponent = default(Component2);
            ModelDoc2 swComponentModel = default(ModelDoc2);
            BomTableAnnotation swBomTableAnnotation = default(BomTableAnnotation);
            BalloonOptions swBomBalloonParams = default(BalloonOptions);
            int i = 0;
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open assembly, create BOM table, and select BOM balloon
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\bladed shaft.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swBomTableAnnotation = (BomTableAnnotation)swModelDocExt.InsertBomTable3("C:\\Program Files\\SolidWorks
            Corp\\SolidWorks\\lang\\english\\bom-standard.sldbomtbt", 618, 568, (int)swBomType_e.swBomType_TopLevelOnly, "", false,
            (int)swNumberingType_e.swNumberingType_None, false);
            status = swModelDocExt.SelectByID2("", "FACE", 0.0139294427590926, 0.059464184169542, 0.0082768338148469, false, 0, null, 0);
            swBomBalloonParams = (BalloonOptions)swModelDocExt.CreateBalloonOptions();
            swBomBalloonParams.Style = (int)swBalloonStyle_e.swBS_SplitCirc;
            swBomBalloonParams.Size = (int)swBalloonFit_e.swBF_Tightest;
            swNote = swModelDocExt.InsertBOMBalloon2(swBomBalloonParams);
            status = swModelDocExt.SelectByID2("DetailItem2@Annotations", "NOTE", 0.00431152692774954, 0.0699703491909496, 0.0033561420724473, false, 0, n
            ull, 0);

            //Get and set BOM balloon properties
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swNote = (Note)swSelectionMgr.GetSelectedObject6(1, -1);
            swAnnotation = (Annotation)swNote.GetAnnotation();
            attachedEntityArray = (object[])swAnnotation.GetAttachedEntities2();
            attachedEntityTypeArray = (int[])swAnnotation.GetAttachedEntityTypes();
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Name = " + swAnnotation.GetName());
            swNote.SetBomBalloonText((int)swDetailingNoteTextContent_e.swDetailingNoteTextCustom, "ABC",
            (int)swDetailingNoteTextContent_e.swDetailingNoteTextCustom, "EFG");
            Debug.Print(" Upper text = " + swNote.GetBomBalloonText(true) + " [" + swNote.GetBomBalloonTextStyle(true) + "]");
            Debug.Print(" Lower text = " + swNote.GetBomBalloonText(false) + " [" + swNote.GetBomBalloonTextStyle(false) + "]");
            Debug.Print(" Balloon fit = " + swNote.GetBalloonSize());
            Debug.Print(" Balloon style = " + swNote.GetBalloonStyle());
            Debug.Print(" Balloon padding = " + swNote.GetBalloonPadding());
            Debug.Print(" Is stacked? " + swNote.IsStackedBalloon());
            Debug.Print(" Is stacked master? " + swNote.IsStackedBalloonMaster());
            for (i = 0; i < attachedEntityArray.Length; i++)
            {
                Debug.Print(" Attached entity type = " + attachedEntityTypeArray[i]);
                if (attachedEntityTypeArray != null)
                {
                    swEntity = (Entity)attachedEntityArray[i];
                    swComponent = (Component2)swEntity.GetComponent();
                    swComponentModel = (ModelDoc2)swComponent.GetComponentModel();
                    Debug.Print(" Attached entity = " + swComponent.GetPathName() + " <" + swComponent.ReferencedConfiguration + ">" + " --
                    > " + swComponentModel.GetPathName());
                }
            }

            swModel.ViewZoomtofit2();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```


Get BOM Balloon Properties Example (VBA)

This example shows how to get the properties of a BOM balloon.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Inserts a BOM table.
' 3. Inserts a BOM balloon.
' 4. Gets and sets the BOM balloon's properties.
' 5. Examine the Immediate window.
'
' NOTE: Because this assembly is used elsewhere, do not save
'       changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swNote As SldWorks.Note
Dim swAnnotation As SldWorks.Annotation
Dim attachedEntityArray As Variant
Dim attachedEntityTypeArray As Variant
Dim swEntity As SldWorks.Entity
Dim swComponent As SldWorks.Component2
Dim swComponentModel As SldWorks.ModelDoc2
Dim swBomTableAnnotation As SldWorks.BomTableAnnotation
Dim swBomBalloonParams As SldWorks.BalloonOptions
Dim i As Long
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open assembly, create BOM table, and select BOM balloon
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swBomTableAnnotation = swModelDocExt.InsertBomTable3("C:\Program Files\SolidWorks Corp\SolidWorks\lang\english\bom-standard.sldbomtbt", 618, 568,
swBomType_e.swBomType_TopLevelOnly, "", False, swNumberingType_e.swNumberingType_None, False)
    status = swModelDocExt.SelectByID2("", "FACE", 1.39294427590926E-02, 0.059464184169542, 8.2768338148469E-03, False, 0, Nothing, 0)
    Set swBomBalloonParams = swModelDocExt.CreateBalloonOptions()
    swBomBalloonParams.Style = swBalloonStyle_e.swBS_SplitCirc
    swBomBalloonParams.Size = swBalloonFit_e.swBF_Tightest
    Set swNote = swModelDocExt.InsertBOMBalloon2(swBomBalloonParams)
    status = swModelDocExt.SelectByID2("DetailItem2@Annotations", "NOTE", 4.31152692774954E-03, 6.99703491909496E-02, 3.3561420724473E-03, False, 0,
Nothing, 0)

    'Get and set BOM balloon properties
    Set swSelectionMgr = swModel.SelectionManager
    Set swNote = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swAnnotation = swNote.GetAnnotation
    attachedEntityArray = swAnnotation.GetAttachedEntities2
    attachedEntityTypeArray = swAnnotation.GetAttachedEntityTypes
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Name = " & swAnnotation.GetName
    swNote.SetBomBalloonText swDetailingNoteTextContent_e.swDetailingNoteTextCustom, "ABC", swDetailingNoteTextContent_e.swDetailingNoteTextCustom, "EFG"
    Debug.Print " Upper text = " & swNote.GetBomBalloonText(True) & " [" & swNote.GetBomBalloonTextStyle(True) & "]"
    Debug.Print " Lower text = " & swNote.GetBomBalloonText(False) & " [" & swNote.GetBomBalloonTextStyle(False) & "]"
    Debug.Print " Balloon fit = " & swNote.GetBalloonSize
    Debug.Print " Balloon style = " & swNote.GetBalloonStyle
    Debug.Print " Balloon padding = " & swNote.GetBalloonPadding
    Debug.Print " Is stacked? " & swNote.IsStackedBalloon
    Debug.Print " Is stacked master? " & swNote.IsStackedBalloonMaster
    For i = 0 To UBound(attachedEntityArray)
        Debug.Print " Attached entity type = " & attachedEntityTypeArray(i)
        If swSelNOTHING <> attachedEntityTypeArray(i) Then
            Set swEntity = attachedEntityArray(i)
            Set swComponent = swEntity.GetComponent
            Set swComponentModel = swComponent.GetModelDoc
            Debug.Print " Attached entity = " & swComponent.GetPathName & " <" & swComponent.ReferencedConfiguration & ">" & " --> " &
swComponentModel.GetPathName
        End If
    Next i

    swModel.ViewZoomtofit2

End Sub

```

Get BOM Balloon Properties Example (VB.NET)

This example shows how to get the properties of a BOM balloon.

```

-----
Preconditions:
1. Verify that the specified assembly document to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified assembly document.
2. Inserts a BOM table.
3. Inserts a BOM balloon.
4. Gets and sets the BOM balloon's properties.
5. Examine the Immediate window.

NOTE: Because this assembly is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swNote As Note
        Dim swAnnotation As Annotation
        Dim attachedEntityArray() As Object
        Dim attachedEntityTypeArray() As Integer
        Dim swEntity As Entity
        Dim swComponent As Component2
        Dim swComponentModel As ModelDoc2
        Dim swBomTableAnnotation As BomTableAnnotation
        Dim swBomBalloonParams As BalloonOptions
        Dim i As Integer
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open assembly, create BOM table, and select BOM balloon
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swBomTableAnnotation = swModelDocExt.InsertBomTable3("C:\Program Files\SolidWorks Corp\SolidWorks\lang\english\bom-standard.sldbomtbt", 618, 568,
        swBomType_e.swBomType_TopLevelOnly, "", False, swNumberingType_e.swNumberingType_None, False)
        status = swModelDocExt.SelectByID("", "FACE", 0.0139294427590926, 0.059464184169542, 0.0082768338148469, False, 0, Nothing, 0)
        swBomBalloonParams = swModelDocExt.CreateBalloonOptions()
        swBomBalloonParams.Style = swBalloonStyle_e.swBS_SplitCirc
        swBomBalloonParams.Size = swBalloonFit_e.swBF_Tightest
        swNote = swModelDocExt.InsertBOMBalloon2(swBomBalloonParams)
        status = swModelDocExt.SelectByID("DetailItem2@Annotations", "NOTE", 0.00431152692774954, 0.0699703491909496, 0.0033561420724473, False, 0, Nothing, 0)

        'Get and set balloon properties
        swSelectionMgr = swModel.SelectionManager
        swNote = swSelectionMgr.GetSelectedObject6(1, -1)
        swAnnotation = swNote.GetAnnotation
        attachedEntityArray = swAnnotation.GetAttachedEntities2
        attachedEntityTypeArray = swAnnotation.GetAttachedEntityTypes
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Name = " & swAnnotation.GetName)
        swNote.SetBomBalloonText(swDetailingNoteTextContent_e.swDetailingNoteTextCustom, "ABC",
        swDetailingNoteTextContent_e.swDetailingNoteTextCustom, "EFG")
        Debug.Print(" Upper text = " & swNote.GetBomBalloonText(True) & " [" & swNote.GetBomBalloonTextStyle(True) & "]")
        Debug.Print(" Lower text = " & swNote.GetBomBalloonText(False) & " [" & swNote.GetBomBalloonTextStyle(False) & "]")
        Debug.Print(" Balloon fit = " & swNote.GetBalloonSize)
        Debug.Print(" Balloon style = " & swNote.GetBalloonStyle)
        Debug.Print(" Balloon padding = " & swNote.GetBalloonPadding)
        Debug.Print(" Is stacked? " & swNote.IsStackedBalloon)
        Debug.Print(" Is stacked master? " & swNote.IsStackedBalloonMaster)
        For i = 0 To UBound(attachedEntityArray)
            Debug.Print(" Attached entity type = " & attachedEntityTypeArray(i))
            If (Not IsNothing(attachedEntityTypeArray(i))) Then
                swEntity = attachedEntityArray(i)
                swComponent = swEntity.GetComponent
                swComponentModel = swComponent.GetModelDoc
                Debug.Print(" Attached entity = " & swComponent.GetPathName & " <" & swComponent.ReferencedConfiguration & ">" & " --")
            End If
        Next i

        swModel.ViewZoomtoFit2

    End Sub

    ''' <summary>
    ''' The Sldworks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Bounding Box Example (C#)

This example shows how to get the bounding boxes for the selected feature and face. This example also shows how to draw 3D sketches depicting the bounding boxes.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects the Sweep1 feature.
// 2. Creates a 3D sketch of the bounding box of
//    the selected Sweep1 feature.
// 3. Selects the top face.
// 4. Creates a 3D sketch of the bounding box of the
//    selected face.
// 5. Examine the graphics area and the Immediate
//    window.
//
// NOTE: Because this part is used elsewhere,
// do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetBoxFeatureFaceCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            Face2 swFace = default(Face2);
            object BoxFeatureArray = null;
            double[] BoxFeatureDblArray = new double[7];
            object BoxFaceArray = null;
            double[] BoxFaceDblArray = new double[7];
            SketchManager swSketchMgr = default(SketchManager);
            SketchPoint[] swSketchPt = new SketchPoint[9];
            SketchSegment[] swSketchSeg = new SketchSegment[13];
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int i = 0;

            // Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cstick.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            // Select feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            status = swModelDocExt.SelectByID("Sweep1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);

            // Get selected feature's bounding box
            Debug.Print("Feature:");
            status = swFeat.GetBox(ref BoxFeatureArray);
            Debug.Assert(status);

            BoxFeatureDblArray = (double[])BoxFeatureArray;

            Debug.Print(" Pt1 = " +
            (" + BoxFeatureDblArray[0] * 1000.0 + ", " + BoxFeatureDblArray[1] * 1000.0 + ", " + BoxFeatureDblArray[2] * 1000.0 + ") mm");
            Debug.Print(" Pt2 = " +
            (" + BoxFeatureDblArray[3] * 1000.0 + ", " + BoxFeatureDblArray[4] * 1000.0 + ", " + BoxFeatureDblArray[5] * 1000.0 + ") mm");
            swModel.Insert3DSketch2(true);
            swModel.SetAddToDB(true);
            swModel.SetDisplayWhenAdded(false);

            swSketchMgr = (SketchManager)swModel.SketchManager;

            // Draw points at each corner of bounding box
            swSketchPt[0] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[3], BoxFeatureDblArray[1], BoxFeatureDblArray[5]);
            swSketchPt[1] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[0], BoxFeatureDblArray[1], BoxFeatureDblArray[5]);
            swSketchPt[2] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[0], BoxFeatureDblArray[1], BoxFeatureDblArray[2]);
            swSketchPt[3] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[3], BoxFeatureDblArray[1], BoxFeatureDblArray[2]);
            swSketchPt[4] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[3], BoxFeatureDblArray[4], BoxFeatureDblArray[5]);
            swSketchPt[5] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[0], BoxFeatureDblArray[4], BoxFeatureDblArray[5]);
            swSketchPt[6] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[0], BoxFeatureDblArray[4], BoxFeatureDblArray[2]);
            swSketchPt[7] = (SketchPoint)swSketchMgr.CreatePoint(BoxFeatureDblArray[3], BoxFeatureDblArray[4], BoxFeatureDblArray[2]);

            // Now draw bounding box
            swSketchSeg[0] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[0].X, swSketchPt[0].Y, swSketchPt[0].Z, swSketchPt[1].X,
            swSketchPt[1].Y, swSketchPt[1].Z);
            swSketchSeg[1] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[1].X, swSketchPt[1].Y, swSketchPt[1].Z, swSketchPt[2].X,
            swSketchPt[2].Y, swSketchPt[2].Z);
```

```

        swSketchSeg[2] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[2].X, swSketchPt[2].Y, swSketchPt[2].Z, swSketchPt[3].X,
swSketchPt[3].Y, swSketchPt[3].Z);
        swSketchSeg[3] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[3].X, swSketchPt[3].Y, swSketchPt[3].Z, swSketchPt[0].X,
swSketchPt[0].Y, swSketchPt[0].Z);
        swSketchSeg[4] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[0].X, swSketchPt[0].Y, swSketchPt[0].Z, swSketchPt[4].X,
swSketchPt[4].Y, swSketchPt[4].Z);
        swSketchSeg[5] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[1].X, swSketchPt[1].Y, swSketchPt[1].Z, swSketchPt[5].X,
swSketchPt[5].Y, swSketchPt[5].Z);
        swSketchSeg[6] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[2].X, swSketchPt[2].Y, swSketchPt[2].Z, swSketchPt[6].X,
swSketchPt[6].Y, swSketchPt[6].Z);
        swSketchSeg[7] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[3].X, swSketchPt[3].Y, swSketchPt[3].Z, swSketchPt[7].X,
swSketchPt[7].Y, swSketchPt[7].Z);
        swSketchSeg[8] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[4].X, swSketchPt[4].Y, swSketchPt[4].Z, swSketchPt[5].X,
swSketchPt[5].Y, swSketchPt[5].Z);
        swSketchSeg[9] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[5].X, swSketchPt[5].Y, swSketchPt[5].Z, swSketchPt[6].X,
swSketchPt[6].Y, swSketchPt[6].Z);
        swSketchSeg[10] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[6].X, swSketchPt[6].Y, swSketchPt[6].Z, swSketchPt[7].X,
swSketchPt[7].Y, swSketchPt[7].Z);
        swSketchSeg[11] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[7].X, swSketchPt[7].Y, swSketchPt[7].Z, swSketchPt[4].X,
swSketchPt[4].Y, swSketchPt[4].Z);

        swModel.SetDisplayWhenAdded(true);
        swModel.SetAddToDB(false);
        swModel.Insert3DSketch2(true);

        swModel.ClearSelection2(true);

        // Get selected face's bounding box
        status = swModelDocExt.SelectByID("", "FACE", -7.62173696102764E-05, 0.2199999999999857, 0.00945327855254163, false, 0, null, 0);
        swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);

        // Get selected face's bounding box
        Debug.Print("Face =");
        BoxFaceArray = (object)swFace.GetBox();

        BoxFaceDblArray = (double[])BoxFaceArray;

        Debug.Print(" Pt1 = " +
(" + BoxFaceDblArray[0] * 1000.0 + ", " + BoxFaceDblArray[1] * 1000.0 + ", " + BoxFaceDblArray[2] * 1000.0 + ") mm");
        Debug.Print(" Pt2 = " +
(" + BoxFaceDblArray[3] * 1000.0 + ", " + BoxFaceDblArray[4] * 1000.0 + ", " + BoxFaceDblArray[5] * 1000.0 + ") mm");

        swModel.Insert3DSketch2(true);
        swModel.SetAddToDB(true);
        swModel.SetDisplayWhenAdded(false);

        // Draw points at each corner of bounding box
        swSketchPt[0] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[3], BoxFaceDblArray[1], BoxFaceDblArray[5]);
        swSketchPt[1] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[0], BoxFaceDblArray[1], BoxFaceDblArray[5]);
        swSketchPt[2] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[0], BoxFaceDblArray[1], BoxFaceDblArray[2]);
        swSketchPt[3] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[3], BoxFaceDblArray[1], BoxFaceDblArray[2]);
        swSketchPt[4] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[3], BoxFaceDblArray[4], BoxFaceDblArray[5]);
        swSketchPt[5] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[0], BoxFaceDblArray[4], BoxFaceDblArray[5]);
        swSketchPt[6] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[0], BoxFaceDblArray[4], BoxFaceDblArray[2]);
        swSketchPt[7] = (SketchPoint)swSketchMgr.CreatePoint(BoxFaceDblArray[3], BoxFaceDblArray[4], BoxFaceDblArray[2]);

        // Now draw bounding box
        swSketchSeg[0] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[0].X, swSketchPt[0].Y, swSketchPt[0].Z, swSketchPt[1].X,
swSketchPt[1].Y, swSketchPt[1].Z);
        swSketchSeg[1] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[1].X, swSketchPt[1].Y, swSketchPt[1].Z, swSketchPt[2].X,
swSketchPt[2].Y, swSketchPt[2].Z);
        swSketchSeg[2] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[2].X, swSketchPt[2].Y, swSketchPt[2].Z, swSketchPt[3].X,
swSketchPt[3].Y, swSketchPt[3].Z);
        swSketchSeg[3] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[3].X, swSketchPt[3].Y, swSketchPt[3].Z, swSketchPt[0].X,
swSketchPt[0].Y, swSketchPt[0].Z);
        swSketchSeg[4] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[0].X, swSketchPt[0].Y, swSketchPt[0].Z, swSketchPt[4].X,
swSketchPt[4].Y, swSketchPt[4].Z);
        swSketchSeg[5] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[1].X, swSketchPt[1].Y, swSketchPt[1].Z, swSketchPt[5].X,
swSketchPt[5].Y, swSketchPt[5].Z);
        swSketchSeg[6] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[2].X, swSketchPt[2].Y, swSketchPt[2].Z, swSketchPt[6].X,
swSketchPt[6].Y, swSketchPt[6].Z);
        swSketchSeg[7] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[3].X, swSketchPt[3].Y, swSketchPt[3].Z, swSketchPt[7].X,
swSketchPt[7].Y, swSketchPt[7].Z);
        swSketchSeg[8] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[4].X, swSketchPt[4].Y, swSketchPt[4].Z, swSketchPt[5].X,
swSketchPt[5].Y, swSketchPt[5].Z);
        swSketchSeg[9] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[5].X, swSketchPt[5].Y, swSketchPt[5].Z, swSketchPt[6].X,
swSketchPt[6].Y, swSketchPt[6].Z);
        swSketchSeg[10] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[6].X, swSketchPt[6].Y, swSketchPt[6].Z, swSketchPt[7].X,
swSketchPt[7].Y, swSketchPt[7].Z);
        swSketchSeg[11] = (SketchSegment)swSketchMgr.CreateLine(swSketchPt[7].X, swSketchPt[7].Y, swSketchPt[7].Z, swSketchPt[4].X,
swSketchPt[4].Y, swSketchPt[4].Z);

        swModel.SetDisplayWhenAdded(true);
        swModel.SetAddToDB(false);
        swModel.Insert3DSketch2(true);
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}

```

Get Bounding Box Example (VBA)

This example shows how to get the bounding boxes for the selected feature and face. This example also shows how to draw 3D sketches depicting the bounding boxes.

```

'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the Sweep1 feature.
' 2. Creates a 3D sketch of the bounding box of
'   the selected Sweep1 feature.
' 3. Selects the top face.
' 4. Creates a 3D sketch of the bounding box of
'   the selected face.
' 5. Examine the graphics area and the Immediate
'   window.
'
' NOTE: Because this part is used elsewhere,
' do not save changes.
'-----

```

Option Explicit

```
Sub main()
```

```

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swModelDocExt        As SldWorks.ModelDocExtension
    Dim swSelMgr              As SldWorks.SelectionMgr
    Dim swFeat               As SldWorks.Feature
    Dim swFace               As SldWorks.Face2
    Dim BoxFeatureArray      As Variant
    Dim BoxFaceArray         As Variant
    Dim swSketchMgr          As SldWorks.SketchManager
    Dim swSketchPt(8)        As SldWorks.SketchPoint
    Dim swSketchSeg(12)      As SldWorks.SketchSegment
    Dim status               As Boolean
    Dim errors               As Long
    Dim warnings             As Long
    Dim fileName             As String

```

```
Set swApp = Application.SldWorks
```

```

' Open part document
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"
Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

```

```

' Select feature
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager
status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)

```

```

' Get selected feature's bounding box
Debug.Print "Feature:"
status = swFeat.GetBox(BoxFeatureArray): Debug.Assert status
Debug.Print " Pt1 = " & _
    "(" & _
    BoxFeatureArray(0) * 1000# & ", " & _
    BoxFeatureArray(1) * 1000# & ", " & _
    BoxFeatureArray(2) * 1000# & _
    ") mm"

```

```

Debug.Print " Pt2 = " & _
    "(" & _
    BoxFeatureArray(3) * 1000# & ", " & _
    BoxFeatureArray(4) * 1000# & ", " & _
    BoxFeatureArray(5) * 1000# & _
    ") mm"

```

```

swModel.Insert3DSketch2 True
swModel.SetAddToDB True
swModel.SetDisplayWhenAdded False

```

```
Set swSketchMgr = swModel.SketchManager
```

```

' Draw points at each corner of bounding box
Set swSketchPt(0) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(1), BoxFeatureArray(5))
Set swSketchPt(1) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(1), BoxFeatureArray(5))
Set swSketchPt(2) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(1), BoxFeatureArray(2))
Set swSketchPt(3) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(1), BoxFeatureArray(2))
Set swSketchPt(4) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(4), BoxFeatureArray(5))
Set swSketchPt(5) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(4), BoxFeatureArray(5))
Set swSketchPt(6) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(4), BoxFeatureArray(2))
Set swSketchPt(7) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(4), BoxFeatureArray(2))

' Now draw bounding box
Set swSketchSeg(0) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z)
Set swSketchSeg(1) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z)
Set swSketchSeg(2) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z)
Set swSketchSeg(3) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z)

Set swSketchSeg(4) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z)
Set swSketchSeg(5) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z)
Set swSketchSeg(6) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z)
Set swSketchSeg(7) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z)

```

```

Set swSketchSeg(8) = swSketchMgr.CreateLine(swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z, swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z)
Set swSketchSeg(9) = swSketchMgr.CreateLine(swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z, swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z)
Set swSketchSeg(10) = swSketchMgr.CreateLine(swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z, swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z)
Set swSketchSeg(11) = swSketchMgr.CreateLine(swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z, swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z)

```

```

swModel.SetDisplayWhenAdded True
swModel.SetAddToDB False
swModel.Insert3DSketch2 True

```

```

swModel.ClearSelection2 True

```

```

' Get selected face's bounding box
status = swModelDocExt.SelectByID("", "FACE", -7.62173696102764E-05, 0.2199999999999857, 9.45327855254163E-03, False, 0, Nothing, 0)
Set swFace = swSelMgr.GetSelectedObject6(1, -1)

```

```

' Get selected face's bounding box
Debug.Print "Face ="
BoxFaceArray = swFace.GetBox
Debug.Print " Pt1 = " & _
    "(" & _
    BoxFaceArray(0) * 1000# & ", " & _
    BoxFaceArray(1) * 1000# & ", " & _
    BoxFaceArray(2) * 1000# & _
    ")" & " mm"
Debug.Print " Pt2 = " & _
    "(" & _
    BoxFaceArray(3) * 1000# & ", " & _
    BoxFaceArray(4) * 1000# & ", " & _
    BoxFaceArray(5) * 1000# & _
    ")" & " mm"

```

```

swModel.Insert3DSketch2 True
swModel.SetAddToDB True
swModel.SetDisplayWhenAdded False

```

```

' Draw points at each corner of bounding box
Set swSketchPt(0) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(1), BoxFaceArray(5))
Set swSketchPt(1) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(1), BoxFaceArray(5))
Set swSketchPt(2) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(1), BoxFaceArray(2))
Set swSketchPt(3) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(1), BoxFaceArray(2))
Set swSketchPt(4) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(4), BoxFaceArray(5))
Set swSketchPt(5) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(4), BoxFaceArray(5))
Set swSketchPt(6) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(4), BoxFaceArray(2))
Set swSketchPt(7) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(4), BoxFaceArray(2))

```

```

' Now draw bounding box
Set swSketchSeg(0) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z)
Set swSketchSeg(1) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z)
Set swSketchSeg(2) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z)
Set swSketchSeg(3) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z)

```

```

Set swSketchSeg(4) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z)
Set swSketchSeg(5) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z)
Set swSketchSeg(6) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z)
Set swSketchSeg(7) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z)

Set swSketchSeg(8) = swSketchMgr.CreateLine(swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z, swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z)
Set swSketchSeg(9) = swSketchMgr.CreateLine(swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z, swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z)
Set swSketchSeg(10) = swSketchMgr.CreateLine(swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z, swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z)
Set swSketchSeg(11) = swSketchMgr.CreateLine(swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z, swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z)

```

```

swModel.SetDisplayWhenAdded True
swModel.SetAddToDB False
swModel.Insert3DSketch2 True

```

```

End Sub

```

Get Bounding Box Example (VB.NET)

This example shows how to get the bounding boxes for the selected feature and face. This example also shows how to draw 3D sketches depicting the bounding boxes.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the Sweep1 feature.
' 2. Creates a 3D sketch of the bounding box of
'   the selected Sweep1 feature.
' 3. Selects the top face.
' 4. Creates a 3D sketch of the bounding box of
'   the selected face.
' 5. Examine the graphics area and the Immediate
'   window.
'
' NOTE: Because this part is used elsewhere,
' do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swFace As Face2
        Dim BoxFeatureArray(6) As Double
        Dim BoxFaceArray(6) As Double
        Dim swSketchMgr As SketchManager
        Dim swSketchPt(8) As SketchPoint
        Dim swSketchSeg(12) As SketchSegment
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        ' Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        ' Select feature
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        status = swModelDocExt.SelectByID2("Sweep1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)

        ' Get selected feature's bounding box
        Debug.Print("Feature:")
        status = swFeat.GetBox(BoxFeatureArray) : Debug.Assert(status)
        Debug.Print(" Pt1 = " & _
            "(" & _
            BoxFeatureArray(0) * 1000.0# & ", " & _
            BoxFeatureArray(1) * 1000.0# & ", " & _
            BoxFeatureArray(2) * 1000.0# & _
            ") mm")
        Debug.Print(" Pt2 = " & _
            "(" & _
            BoxFeatureArray(3) * 1000.0# & ", " & _
            BoxFeatureArray(4) * 1000.0# & ", " & _
            BoxFeatureArray(5) * 1000.0# & _
            ") mm")

        swModel.Insert3DSketch2(True)
        swModel.SetAddToDB(True)
        swModel.SetDisplayWhenAdded(False)

        swSketchMgr = swModel.SketchManager

        ' Draw points at each corner of bounding box
        swSketchPt(0) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(1), BoxFeatureArray(5))
        swSketchPt(1) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(1), BoxFeatureArray(5))
        swSketchPt(2) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(1), BoxFeatureArray(2))
        swSketchPt(3) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(1), BoxFeatureArray(2))
        swSketchPt(4) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(4), BoxFeatureArray(5))
        swSketchPt(5) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(4), BoxFeatureArray(5))
        swSketchPt(6) = swSketchMgr.CreatePoint(BoxFeatureArray(0), BoxFeatureArray(4), BoxFeatureArray(2))
        swSketchPt(7) = swSketchMgr.CreatePoint(BoxFeatureArray(3), BoxFeatureArray(4), BoxFeatureArray(2))

        ' Now draw bounding box
        swSketchSeg(0) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(1).X, swSketchPt(1).Y,
        swSketchPt(1).Z)
        swSketchSeg(1) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(2).X, swSketchPt(2).Y,
        swSketchPt(2).Z)
        swSketchSeg(2) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(3).X, swSketchPt(3).Y,
        swSketchPt(3).Z)
```

```

        swSketchSeg(3) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(0).X, swSketchPt(0).Y,
swSketchPt(0).Z)
        swSketchSeg(4) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(4).X, swSketchPt(4).Y,
swSketchPt(4).Z)
        swSketchSeg(5) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(5).X, swSketchPt(5).Y,
swSketchPt(5).Z)
        swSketchSeg(6) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(6).X, swSketchPt(6).Y,
swSketchPt(6).Z)
        swSketchSeg(7) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(7).X, swSketchPt(7).Y,
swSketchPt(7).Z)
        swSketchSeg(8) = swSketchMgr.CreateLine(swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z, swSketchPt(5).X, swSketchPt(5).Y,
swSketchPt(5).Z)
        swSketchSeg(9) = swSketchMgr.CreateLine(swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z, swSketchPt(6).X, swSketchPt(6).Y,
swSketchPt(6).Z)
        swSketchSeg(10) = swSketchMgr.CreateLine(swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z, swSketchPt(7).X, swSketchPt(7).Y,
swSketchPt(7).Z)
        swSketchSeg(11) = swSketchMgr.CreateLine(swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z, swSketchPt(4).X, swSketchPt(4).Y,
swSketchPt(4).Z)

swModel.SetDisplayWhenAdded(True)
swModel.SetAddToDB(False)
swModel.Insert3DSketch2(True)

swModel.ClearSelection2(True)

' Get selected face's bounding box
status = swModelDocExt.SelectByID("", "FACE", -0.0000762173696102764, 0.2199999999999857, 0.00945327855254163, False, 0, Nothing, 0)
swFace = swSelMgr.GetSelectedObject6(1, -1)

' Get selected face's bounding box
Debug.Print("Face =")
BoxFaceArray = swFace.GetBox
Debug.Print(" Pt1 = " & _
"(" & _
BoxFaceArray(0) * 1000.0# & ", " & _
BoxFaceArray(1) * 1000.0# & ", " & _
BoxFaceArray(2) * 1000.0# & _
") mm")
Debug.Print(" Pt2 = " & _
"(" & _
BoxFaceArray(3) * 1000.0# & ", " & _
BoxFaceArray(4) * 1000.0# & ", " & _
BoxFaceArray(5) * 1000.0# & _
") mm")

swModel.Insert3DSketch2(True)
swModel.SetAddToDB(True)
swModel.SetDisplayWhenAdded(False)

' Draw points at each corner of bounding box
swSketchPt(0) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(1), BoxFaceArray(5))
swSketchPt(1) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(1), BoxFaceArray(5))
swSketchPt(2) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(1), BoxFaceArray(2))
swSketchPt(3) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(1), BoxFaceArray(2))
swSketchPt(4) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(4), BoxFaceArray(5))
swSketchPt(5) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(4), BoxFaceArray(5))
swSketchPt(6) = swSketchMgr.CreatePoint(BoxFaceArray(0), BoxFaceArray(4), BoxFaceArray(2))
swSketchPt(7) = swSketchMgr.CreatePoint(BoxFaceArray(3), BoxFaceArray(4), BoxFaceArray(2))

' Now draw bounding box
swSketchSeg(0) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(1).X, swSketchPt(1).Y,
swSketchPt(1).Z)
swSketchSeg(1) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(2).X, swSketchPt(2).Y,
swSketchPt(2).Z)
swSketchSeg(2) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(3).X, swSketchPt(3).Y,
swSketchPt(3).Z)
swSketchSeg(3) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(0).X, swSketchPt(0).Y,
swSketchPt(0).Z)
swSketchSeg(4) = swSketchMgr.CreateLine(swSketchPt(0).X, swSketchPt(0).Y, swSketchPt(0).Z, swSketchPt(4).X, swSketchPt(4).Y,
swSketchPt(4).Z)
swSketchSeg(5) = swSketchMgr.CreateLine(swSketchPt(1).X, swSketchPt(1).Y, swSketchPt(1).Z, swSketchPt(5).X, swSketchPt(5).Y,
swSketchPt(5).Z)
swSketchSeg(6) = swSketchMgr.CreateLine(swSketchPt(2).X, swSketchPt(2).Y, swSketchPt(2).Z, swSketchPt(6).X, swSketchPt(6).Y,
swSketchPt(6).Z)
swSketchSeg(7) = swSketchMgr.CreateLine(swSketchPt(3).X, swSketchPt(3).Y, swSketchPt(3).Z, swSketchPt(7).X, swSketchPt(7).Y,
swSketchPt(7).Z)
swSketchSeg(8) = swSketchMgr.CreateLine(swSketchPt(4).X, swSketchPt(4).Y, swSketchPt(4).Z, swSketchPt(5).X, swSketchPt(5).Y,
swSketchPt(5).Z)
swSketchSeg(9) = swSketchMgr.CreateLine(swSketchPt(5).X, swSketchPt(5).Y, swSketchPt(5).Z, swSketchPt(6).X, swSketchPt(6).Y,
swSketchPt(6).Z)
swSketchSeg(10) = swSketchMgr.CreateLine(swSketchPt(6).X, swSketchPt(6).Y, swSketchPt(6).Z, swSketchPt(7).X, swSketchPt(7).Y,
swSketchPt(7).Z)
swSketchSeg(11) = swSketchMgr.CreateLine(swSketchPt(7).X, swSketchPt(7).Y, swSketchPt(7).Z, swSketchPt(4).X, swSketchPt(4).Y,
swSketchPt(4).Z)

swModel.SetDisplayWhenAdded(True)
swModel.SetAddToDB(False)
swModel.Insert3DSketch2(True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Break Line Data Example (C#)

This example shows how to get information on all break lines in a view.

```
//-----
// Preconditions:
// 1. Create or open a drawing with one or more views that contain
//    one or more break lines.
// 2. Open an Immediate Window.
//
// Postconditions: Inspect the Immediate Window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetBreakline_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        DrawingDoc Draw;
        View swActiveView;
        int Size;
        int Breaks;
        object info;
        Double[] infoArray;

        public void Main()
        {
            Draw = (DrawingDoc)swApp.ActiveDoc;
            long count = 0;
            count = Draw.GetViewCount();
            Debug.Print("There are " + count + " views in this drawing including the sheet view.");
            long i = 0;
            long j = 0;
            swActiveView = (View)Draw.GetFirstView();
            // get the sheet
            swActiveView = (View)swActiveView.GetNextView();
            // get the first view
            for (i = 0; i <= count - 2; i++)
            {
                Debug.Print("View " + (i + 1));
                Breaks = swActiveView.GetBreakLineCount2(out Size);
                Debug.Print("    Number of breaks is " + Breaks);
                Debug.Print("    Size of break line data array is " + Size);

                if (!(Breaks == 0))
                {
                    info = swActiveView.GetBreakLineInfo2();
                    infoArray = (Double[])info;

                    // General break line information
                    Debug.Print("    General break line info:");
                    Debug.Print("        Style as defined in swBreakLineStyle_e: " + infoArray[0]);
                    Debug.Print("        Color (0 or -1 for default color): " + infoArray[1]);
                    Debug.Print("        Line type as defined in swLineTypes_e: " + infoArray[2]);
                    Debug.Print("        Line style as defined in swLineStyles_e: " + infoArray[3]);
                    Debug.Print("        Line weight as defined in swLineWeights_e: " + infoArray[4]);
                    Debug.Print("        Layer id: " + infoArray[5]);
                    Debug.Print("        Layer override as defined in swLayerOverride_e: " + infoArray[6]);
                    Debug.Print("        Number of line segments: " + infoArray[7]);
                    Debug.Print("        Number of arcs: " + infoArray[8]);
                    Debug.Print("        Number of jagged lines: " + infoArray[9]);

                    // Straight, zigzag, arc, or jagged break line data
                    Debug.Print("");
                    Debug.Print("    Straight, zigzag, arc, or jagged break line data");

                    for (j = 10; j <= Size - 1; j++)
                    {
                        Debug.Print("        " + infoArray[j]);
                    }

                    swActiveView = (View)swActiveView.GetNextView();
                }
            }
        }

        public SldWorks swApp;
    }
}
```

Get Break Line Data Example (VBA)

This example shows how to get information on all break lines in a view.

```

'-----
' Preconditions:
' 1. Create or open a drawing with one or more views that contain
'   one or more break lines.
' 2. Open an Immediate Window.
'
' Postconditions: Inspect the Immediate Window.
'-----
Dim swApp As SldWorks.SldWorks
Dim Draw As SldWorks.DrawingDoc
Dim swActiveView As SldWorks.View
Dim Size As Long
Dim Breaks As Long
Dim info As Variant

Option Explicit

Sub main()
    Set swApp = Application.SldWorks
    Set Draw = swApp.ActiveDoc
    Dim count As Long
    count = Draw.GetViewCount
    Debug.Print "There are " & count & " views in this drawing including the sheet view."
    Dim i As Long
    Dim j As Long
    Set swActiveView = Draw.GetFirstView ' get the sheet
    Set swActiveView = swActiveView.GetNextView ' get the first view
    For i = 0 To count - 2
        Debug.Print "View " & i + 1
        Breaks = swActiveView.GetBreakLineCount2(Size)
        Debug.Print "    Number of breaks is " & Breaks
        Debug.Print "    Size of break line data array is " & Size

        If Not Breaks = 0 Then
            info = swActiveView.GetBreakLineInfo2
            ' General break line information
            Debug.Print "    Break line info"
            Debug.Print "        Style as defined in swBreakLineStyle_e: " & info(0)
            Debug.Print "        Color (0 or -1 for default color): " & info(1)
            Debug.Print "        Line type as defined in swLineTypes_e: " & info(2)
            Debug.Print "        Line style as defined in swLineStyle_e: " & info(3)
            Debug.Print "        Line weight as defined in swLineWeights_e: " & info(4)
            Debug.Print "        Layer id: " & info(5)
            Debug.Print "        Layer override as defined in swLayerOverride_e: " & info(6)
            Debug.Print "        Number of line segments: " & info(7)
            Debug.Print "        Number of arcs: " & info(8)
            Debug.Print "        Number of jagged lines: " & info(9)

            ' Straight, zigzag, arc, or jagged break line data
            Debug.Print ""
            Debug.Print "        Straight, zigzag, arc, or jagged break line data:"
            For j = 10 To Size - 1
                Debug.Print "            " & info(j)
            Next j

        End If
        Set swActiveView = swActiveView.GetNextView
    Next i
End Sub

```

Get Break Line Data Example (VB.NET)

This example shows how to get information on all break lines in a view.

```

'-----
' Preconditions:
' 1. Create or open a drawing with one or more views that contain
'    one or more break lines.
' 2. Open an Immediate Window.
'-----
' Postconditions: Inspect the Immediate Window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Draw As DrawingDoc
    Dim swActiveView As View
    Dim Size As Long
    Dim Breaks As Long
    Dim info As Object
    Dim boolstatus As Boolean

    Sub main()

        Draw = swApp.ActiveDoc
        Dim count As Long
        count = Draw.GetViewCount
        Debug.Print("There are " & count & " views in this drawing including the sheet view.")
        Dim i As Long
        Dim j As Long
        swActiveView = Draw.GetFirstView ' get the sheet
        swActiveView = swActiveView.GetNextView ' get the first view
        For i = 0 To count - 2
            Debug.Print("View " & i + 1)
            Breaks = swActiveView.GetBreakLineCount2(Size)
            Debug.Print("    Number of breaks is " & Breaks)
            Debug.Print("    Size of break line data array is " & Size)

            If Not Breaks = 0 Then
                info = swActiveView.GetBreakLineInfo2

                ' General break line information
                Debug.Print("    General break line info")
                Debug.Print("        Style as defined in swBreakLineStyle_e: " & info(0))
                Debug.Print("        Color (0 or -1 for default color): " & info(1))
                Debug.Print("        Line type as defined in swLineTypes_e: " & info(2))
                Debug.Print("        Line style as defined in swLineStyle_e: " & info(3))
                Debug.Print("        Line weight as defined in swLineWeights_e: " & info(4))
                Debug.Print("        Layer id: " & info(5))
                Debug.Print("        Layer override as defined in swLayerOverride_e: " & info(6))
                Debug.Print("        Number of line segments: " & info(7))
                Debug.Print("        Number of arcs: " & info(8))
                Debug.Print("        Number of jagged lines: " & info(9))

                ' Straight, zigzag, arc, or jagged break line data
                Debug.Print("")
                Debug.Print("        Straight, zigzag, arc, or jagged break line data:")
                For j = 10 To Size - 1
                    Debug.Print("            " & info(j))
                Next j

            End If
            swActiveView = swActiveView.GetNextView
        Next i

    End Sub

    Public swApp As SldWorks

End Class

```

Get Broken-Out Section Feature Data Example (C#)

This example shows how to get broken-out section feature data from a drawing view.

```
//-----
// Preconditions: Open a drawing that contains a view with a broken-out
// section feature.
//
// Postconditions:
// 1. Inspect the Immediate window.
// 2. The radius of the broken-out section is increased.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace GetBrokenOutSections_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swDoc;
        DrawingDoc swDDoc;
        View swView;
        object[] varSheet;
        object[] varViews;
        int I;
        int J;
        int K;
        Feature swFeat;
        BrokenOutSectionFeatureData swFeatData;
        Entity swDepthRef;
        object[] varBS;
        bool doRelease;
        object[] varskSeg;
        int segCtr;
        int segType;

        public void Main()
        {
            swDoc = (ModelDoc2)swApp.ActiveDoc;
            swDDoc = (DrawingDoc)swDoc;
            swDoc.ClearSelection2(true);
            varSheet = (object[])swDDoc.GetViews();
            for (I = varSheet.GetLowerBound(0); I <= varSheet.GetUpperBound(0); I++)
            {
                varViews = (object[])varSheet[I];
                // skip the sheet view
                for (J = 1; J <= varViews.GetUpperBound(0); J++)
                {
                    View swView = default(View);
                    swView = (View)varViews[J];
                    Debug.Print("View: " + swView.Name);
                    Debug.Print(" Number of broken-out sections in this view: " + swView.GetBreakOutSectionCount());
                    varBS = (object[])swView.GetBreakOutSections();
                    if (!((varBS == null)))
                    {
                        for (K = varBS.GetLowerBound(0); K <= varBS.GetUpperBound(0); K++)
                        {
                            swFeat = (Feature)varBS[K];
                            EditFeature(swFeat);
                            swFeatData = (BrokenOutSectionFeatureData)swFeat.GetDefinition();
                            swFeatData.AccessSelections(swDoc, null);
                            swDepthRef = swFeatData.DepthReference;
                            // get and select depth reference, if set
                            if ((swDepthRef == null))
                            {
                                Debug.Print("          Depth of exposure in meters: " + swFeatData.Depth);
                            }
                            else
                            {
                                swDepthRef.Select(true);
                            }
                            swFeatData.ReleaseSelectionAccess();
                            swFeatData = null;
                        }
                    }
                }
            }
        }

        public void GetExistingValue()
        {
            swFeatData.EditSketch = true;
            varskSeg = (object[])swFeatData.SketchSegment;
            if (!((varskSeg == null)))
            {

```

```

        Debug.Print("                Number of sketch segments: " + swFeatData.GetSketchSegmentCount());
    }
}
public void EditFeature(Feature inFeat)
{
    Debug.Print("            Iterating on broken-out section: " + inFeat.Name);

    swFeatData = (BrokenOutSectionFeatureData)inFeat.GetDefinition();
    swFeatData.AccessSelections(swDoc, null);

    GetExistingValue();

    if ((doRelease == false))
    {
        if (((!(varskSeg == null))))
        {
            for (segCtr = varskSeg.GetLowerBound(0); segCtr <= varskSeg.GetUpperBound(0); segCtr++)
            {
                SketchSegment swSeg = default(SketchSegment);
                swSeg = (SketchSegment)varskSeg[segCtr];
                segType = swSeg.GetType();
                switch (segType)
                {
                    case (int)swSketchSegments_e.swSketchLINE:
                        SketchLine swSegLine = default(SketchLine);
                        swSegLine = (SketchLine)swSeg;
                        Debug.Print("                Segment type: Line");
                        swSegLine = null;
                        break;
                    case (int)swSketchSegments_e.swSketchARC:
                        SketchArc swSegArc = default(SketchArc);
                        swSegArc = (SketchArc)swSeg;
                        Debug.Print("                Segment type: Arc");
                        Debug.Print("                Old radius: " + swSegArc.GetRadius());
                        double newRad = 0;
                        newRad = swSegArc.GetRadius() + 0.005;
                        Debug.Print("                New radius: " + newRad);
                        swSegArc.SetRadius(newRad);
                        swSegArc = null;
                        break;
                    default:
                        Debug.Print("                Segment type: Unknown");
                        break;
                }
            }
        }
        inFeat.ModifyDefinition(swFeatData, swDoc, null);
        swDoc.ClearSelection();
    }
    else
    {
        swFeatData.ReleaseSelectionAccess();
    }
    varskSeg = null;
    swFeatData = null;
}

public SldWorks swApp;
}
}

```

Get Broken-Out Section Feature Data Example (VBA)

This example shows how to get broken-out section feature data from a drawing view.

```

'-----
' Preconditions: Open a drawing that contains a view with a broken-out
' section feature.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The radius of the broken-out section is increased.
'-----

Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swDDoc As SldWorks.DrawingDoc
Dim varSheet As Variant
Dim varViews As Variant
Dim I As Long
Dim J As Long
Dim K As Long
Dim swFeat As SldWorks.Feature
Dim swFeatData As SldWorks.BrokenOutSectionFeatureData
Dim swDepthRef As SldWorks.Entity
Dim varBS As Variant
Dim doRelease As Boolean
Dim varskSeg As Variant
Dim segCtr As Long
Dim segType As Long

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set swDDoc = swDoc
    swDoc.ClearSelection2 True
    varSheet = swDDoc.GetViews
    For I = LBound(varSheet) To UBound(varSheet)
        varViews = varSheet(I)
        For J = 1 To UBound(varViews) ' skip the sheet view
            Dim swView As SldWorks.View
            Set swView = varViews(J)
            Debug.Print "View: " & swView.Name
            Debug.Print (" Number of broken-out sections in this view: " & swView.GetBreakOutSectionCount)
            varBS = swView.GetBreakOutSections
            If (Not (IsEmpty(varBS))) Then
                For K = LBound(varBS) To UBound(varBS)
                    Set swFeat = varBS(K)
                    Call EditFeature(swFeat)
                    Set swFeatData = swFeat.GetDefinition
                    swFeatData.AccessSelections swDoc, Nothing
                    Set swDepthRef = swFeatData.DepthReference ' depth reference is not set
                    If (swDepthRef Is Nothing) Then
                        Debug.Print ("          Depth of exposure in meters: " & swFeatData.Depth)
                    Else
                        swDepthRef.Select True
                    End If
                    swFeatData.ReleaseSelectionAccess
                    Set swFeatData = Nothing
                Next K
            End If
        Next J
    Next I
End Sub

Sub GetExistingValue()
    swFeatData.EditSketch = True
    varskSeg = swFeatData.SketchSegment
    If (Not (IsEmpty(varskSeg))) Then
        Debug.Print ("          Number of sketch segments: " & swFeatData.GetSketchSegmentCount)
    End If
End Sub

Sub EditFeature(inFeat As SldWorks.Feature)
    Debug.Print ("          Iterating on broken-out section: " & inFeat.Name)

    Set swFeatData = inFeat.GetDefinition
    swFeatData.AccessSelections swDoc, Nothing

    GetExistingValue

    If (doRelease = False) Then
        If (Not (IsEmpty(varskSeg))) Then
            For segCtr = LBound(varskSeg) To UBound(varskSeg)
                Dim swSeg As SldWorks.SketchSegment
                Set swSeg = varskSeg(segCtr)
            Next segCtr
        End If
    End If
End Sub

```

```

segType = swSeg.GetType
Select Case segType
    Case swSketchLINE
        Dim swSegLine As SldWorks.SketchLine
        Set swSegLine = swSeg
        Debug.Print ("                Segment type: Line")
        Set swSegLine = Nothing
    Case swSketchARC
        Dim swSegArc As SldWorks.SketchArc
        Set swSegArc = swSeg
        Debug.Print ("                Segment type: Arc")
        Debug.Print ("                Old radius: " & swSegArc.GetRadius)
        Dim newRad As Double
        newRad = swSegArc.GetRadius + 0.005
        Debug.Print ("                New radius: " & newRad)
        swSegArc.SetRadius (newRad)
        Set swSegArc = Nothing
    Case Else
        Debug.Print ("                Segment type: Unknown")
End Select
Next segCtr
End If
inFeat.ModifyDefinition swFeatData, swDoc, Nothing
swDoc.ClearSelection
Else
    swFeatData.ReleaseSelectionAccess
End If

varskSeg = Empty
Set swFeatData = Nothing

End Sub

```

Get Broken-Out Section Feature Data Example (VB.NET)

This example shows how to get broken-out section feature data from a drawing view.

Preconditions: Open a drawing that contains a view with a broken-out section feature.

Postconditions:
1. Inspect the Immediate window.
2. The radius of the broken-out section is increased.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swDoc As ModelDoc2
    Dim swDDoc As DrawingDoc
    Dim swView As View
    Dim varSheet As Object
    Dim varViews As Object
    Dim I As Long
    Dim J As Long
    Dim K As Long
    Dim swFeat As Feature
    Dim swFeatData As BrokenOutSectionFeatureData
    Dim swDepthRef As Entity
    Dim varBS As Object
    Dim doRelease As Boolean
    Dim varskSeg As Object
    Dim segCtr As Long
    Dim segType As Long

    Sub main()
        swDoc = swApp.ActiveDoc
        swDDoc = swDoc
        swDoc.ClearSelection2(True)
        varSheet = swDDoc.GetViews
        For I = LBound(varSheet) To UBound(varSheet)
            varViews = varSheet(I)
            For J = 1 To UBound(varViews) ' skip the sheet view
                Dim swView As View
                swView = varViews(J)
                Debug.Print("View: " & swView.Name)
                Debug.Print(" Number of broken-out sections in this view: " & swView.GetBreakOutSectionCount)
                varBS = swView.GetBreakOutSections
                If (Not (IsNothing(varBS))) Then
                    For K = LBound(varBS) To UBound(varBS)
                        swFeat = varBS(K)
                        Call EditFeature(swFeat)
                        swFeatData = swFeat.GetDefinition
                        swFeatData.AccessSelections(swDoc, Nothing)
                        swDepthRef = swFeatData.DepthReference ' get and select depth reference, if set
                        If (swDepthRef Is Nothing) Then
                            Debug.Print(" Depth of exposure in meters: " & swFeatData.Depth)
                        Else
                            swDepthRef.Select(True)
                        End If
                        swFeatData.ReleaseSelectionAccess()
                        swFeatData = Nothing
                    Next K
                End If
            Next J
        Next I
    End Sub

    Sub GetExistingValue()
        swFeatData.EditSketch = True
        varskSeg = swFeatData.SketchSegment
        If (Not (IsNothing(varskSeg))) Then
            Debug.Print(" Number of sketch segments: " & swFeatData.GetSketchSegmentCount)
        End If
    End Sub

    Sub EditFeature(ByVal inFeat As Feature)
        Debug.Print(" Iterating on broken-out section: " & inFeat.Name)

        swFeatData = inFeat.GetDefinition
        swFeatData.AccessSelections(swDoc, Nothing)

        GetExistingValue()

        If (doRelease = False) Then
            If (Not (IsNothing(varskSeg))) Then
                For segCtr = LBound(varskSeg) To UBound(varskSeg)
```

```

Dim swSeg As SketchSegment
swSeg = varskSeg(segCtr)
segType = swSeg.GetType
Select Case segType
    Case swSketchSegments_e.swSketchLINE
        Dim swSegLine As SketchLine
        swSegLine = swSeg
        Debug.Print("                Segment type: Line")
        swSegLine = Nothing
    Case swSketchSegments_e.swSketchARC
        Dim swSegArc As SketchArc
        swSegArc = swSeg
        Debug.Print("                Segment type: Arc")
        Debug.Print("                Old radius: " & swSegArc.GetRadius)
        Dim newRad As Double
        newRad = swSegArc.GetRadius + 0.005
        Debug.Print("                New radius: " & newRad)
        swSegArc.SetRadius(newRad)
        swSegArc = Nothing
    Case Else
        Debug.Print("                Segment type: Unknown")
End Select
Next segCtr
End If
inFeat.ModifyDefinition(swFeatData, swDoc, Nothing)
swDoc.ClearSelection()
Else
    swFeatData.ReleaseSelectionAccess()
End If
varskSeg = Nothing
swFeatData = Nothing
End Sub

Public swApp As SldWorks

End Class

```

Get Build Numbers Example (C#)

This example shows how to get the build, major revision, and hot fix numbers of the SOLIDWORKS Design application.

```
//-----  
// Preconditions: Open SOLIDWORKS Design.  
//  
// Postconditions: Inspect the Immediate window for build,  
// major revision, and hot fix numbers.  
//-----  
  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
namespace GetBuildNumbers2_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        string BaseVersion;  
        string CurrentVersion;  
        string HotFixes;  
  
        public void Main()  
        {  
            swApp.GetBuildNumbers2(out BaseVersion, out CurrentVersion, out HotFixes);  
            Debug.Print("SOLIDWORKS major revision number: " + BaseVersion);  
            Debug.Print("SOLIDWORKS build number: " + CurrentVersion);  
            Debug.Print("SOLIDWORKS hot fix numbers: " + HotFixes);  
        }  
    }  
    public SldWorks swApp;  
}
```

Get Build Numbers Example (VBA)

This example shows how to get the build, major revision, and hot fix numbers of the SOLIDWORKS Design application.

```
'-----  
' Preconditions: Open SOLIDWORKS Design.  
'  
' Postconditions: Inspect the Immediate window for build,  
' major revision, and hot fix numbers.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim BaseVersion As String  
Dim CurrentVersion As String  
Dim HotFixes As String
```

```
Option Explicit
```

```
Sub main()  
    Set swApp = Application.SldWorks  
    swApp.GetBuildNumbers2 BaseVersion, CurrentVersion, HotFixes  
    Debug.Print "SOLIDWORKS major revision number: " & BaseVersion  
    Debug.Print "SOLIDWORKS build number: " & CurrentVersion  
    Debug.Print "SOLIDWORKS hot fix numbers: " & HotFixes  
End Sub
```

Get Build Numbers Example (VB.NET)

This example shows how to get the build, major revision, and hot fix numbers of the SOLIDWORKS Design application.

```
'-----  
' Preconditions: Open SOLIDWORKS Design.  
'  
' Postconditions: Inspect the Immediate window for build,  
' major revision, and hot fix numbers.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim BaseVersion As String  
    Dim CurrentVersion As String  
    Dim HotFixes As String  
  
    Sub main()  
  
        swApp.GetBuildNumbers2(BaseVersion, CurrentVersion, HotFixes)  
        Debug.Print("SOLIDWORKS major revision number: " & BaseVersion)  
        Debug.Print("SOLIDWORKS build number: " & CurrentVersion)  
        Debug.Print("SOLIDWORKS hot fix numbers: " & HotFixes)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Get Centerline Annotation Information Example (VBA)

This example shows how to get information about a centerline annotation, which is stored in its IDisplayData object. You get it using:

- ISelectionManager::GetSelectedObject6
- ICenterline::GetAnnotation
- IAnnotation::GetDisplayData
- IDisplayData::GetLineAtIndex3

```
'-----  
'  
' Preconditions: Drawing document is open  
'           and a centerline is selected.  
'  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swSelAnn As SldWorks.Annotation  
Dim swDisplayData As SldWorks.DisplayData  
Dim swSelCenterLine As SldWorks.CenterLine  
Dim params As Variant  
Dim i As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swSelCenterLine = swSelMgr.GetSelectedObject6(1, 0)  
  
Set swSelAnn = swSelCenterLine.GetAnnotation  
Set swDisplayData = swSelAnn.GetDisplayData  
Debug.Print "Centerline annotation information: "  
For i = 0 To 9  
    params = swDisplayData.GetLineAtIndex3(0)  
    Debug.Print params(i)  
Next i  
End Sub
```